

STM32 Gömülü Sistemler

ve ARM Assembly Rehberi

Register Seviyesinden Assembly'ye Uygulamalı Başvuru

Bu belge, STM32 (ARM Cortex-M) mikrodnetleyicilerini **register seviyesinden** başlayarak, çevre birimleri, kesmeler ve DMA ile **çalışan kod örnekleriyle** anlatır; ardından donanımın altındaki **ARM Cortex-M assembly** dilini komut komut öğretir.

Kısım 1 — STM32 & Cortex-M Temelleri: mimari • bellek haritası • saat (clock) sistemi • CMSIS • araç zinciri.

Kısım 2 — GPIO ve Register Programlama: register erişimi • bit işlemleri • LED yakma • buton okuma • bare-metal vs HAL.

Kısım 3 — Çevre Birimleri: zamanlayıcı (timer) & PWM • UART • ADC • SPI & I2C.

Kısım 4 — Kesmeler: NVIC • EXTI • SysTick • kesme öncelikleri • DMA.

Kısım 5 — ARM Assembly Temelleri: Cortex-M programlama modeli • register'lar • Thumb-2 • temel komutlar.

Kısım 6 — ARM Assembly İleri: akış kontrolü • fonksiyonlar & yığın (AAPCS) • diziler • C ile karıştırma.

Hazırlanma Tarihi: 13 Temmuz 2026

STM32 • ARM Cortex-M • Kodlar İngilizce, açıklamalar Türkçe

Contents

1 Giriş: Mikrodenetleyiciler ve STM32	3
1.1 Mikroişlemci vs Mikrodenetleyici	3
1.2 STM32 Ailesi ve Cortex-M Çekirdekleri	3
1.3 Neden Register Seviyesi Öğrenmeli?	3
2 Bellek Haritası ve Register Modeli	4
2.1 Cortex-M Bellek Haritası	4
2.2 Register'a Erişim: volatile İşaretçiler	4
2.3 CMSIS: Standart Register Tanımları	5
3 Saat Sistemi ve Araç Zinciri	5
3.1 Saat (Clock) Sistemi ve RCC	5
3.2 Araç Zinciri ve Derleme	5
4 Saat Ağacı (Clock Tree) Detayı	6
4.1 Saat Kaynakları ve PLL	6
5 GPIO: Genel Amaçlı Giriş/Çıkış	8
5.1 GPIO Register'ları	8
5.2 Bit İşlemleri: Register Programlamının Dili	8
6 İlk Program: LED Yakma (Blink)	9
6.1 Bare-Metal LED Blink	9
6.2 Aynı İş HAL ile	9
7 Giriş Okuma: Buton	10
8 Zamanlayıcılar (Timers) ve PWM	12
8.1 Zamanlayıcı Nasıl Çalışır?	12
8.2 PWM Üretme	12
8.3 HAL Driver ile Timer ve PWM	13
9 Seri İletişim: UART	15
9.1 UART Yapılandırma ve Gönderme	15
9.2 HAL Driver ile UART	15
10 ADC ve Diğer Veri Yolları (SPI, I2C)	17
10.1 ADC: Analog Okuma	17
10.2 HAL Driver ile ADC	18
10.3 SPI ve I2C: Sensör Veri Yolları	19
11 Kesmeler ve NVIC	22
11.1 NVIC: Kesme Denetleyicisi	22
11.2 Timer Kesmesi Örneği	22
11.3 Harici Kesme (EXTI): Buton ile	23
11.4 HAL Driver ile Harici Kesme	23
12 SysTick ve Zamanlama	24
13 DMA: İşlemciyi Meşgul Etmeden Veri Taşıma	25
14 RTC: Gerçek Zaman Saati	26

15 Watchdog: Sistemi Kilitlenmeden Koruma	27
16 Düşük Güç Modları	27
17 CRC ve Diğer Sistem Birimleri	28
18 Hata Ayıklama ve İzleme (Debugging)	29
18.1 SWO ile printf İzleme (ITM)	29
18.2 HardFault Ayıklama	30
19 FreeRTOS: Gerçek Zamanlı İşletim Sistemi	30
19.1 Görevler (Tasks)	30
19.2 Görevler Arası İletişim: Kuyruk ve Semafor	31
20 ARM Mimarisi ve Programlama Modeli	33
20.1 RISC ve Load/Store Mimarisi	33
20.2 Register'lar (Cortex-M Programlama Modeli)	33
20.3 Durum Bayrakları (Condition Flags)	34
20.4 Thumb ve Thumb-2	34
21 Temel Komutlar: Veri ve Aritmetik	34
21.1 Veri Taşıma: MOV ve LDR	34
21.2 Bellek Erişimi: LDR ve STR	34
21.3 Aritmetik ve Mantık Komutları	35
22 Akış Kontrolü: Dallanma ve Döngüler	36
22.1 Karşılaştırma ve Koşullu Dallanma	36
22.2 if Yapısı	36
22.3 Döngü: Bir Diziyi Toplama	36
23 Fonksiyonlar, Yığın ve Çağrı Kuralı (AAPCS)	37
23.1 Çağrı Kuralı (Calling Convention)	37
23.2 Basit Fonksiyon (Leaf Function)	37
23.3 Yığın (Stack): PUSH ve POP	38
23.4 Özyinelemeli Fonksiyon: Faktöriyel	38
24 Donanım, Startup ve C ile Karıştırma	38
24.1 Assembly'den GPIO Register'ına Yazma	38
24.2 C ile Inline Assembly	38
24.3 Cortex-M Startup Kodu ve Vektör Tablosu	39
25 Barrel Shifter ve Adresleme Modları	40
25.1 Barrel Shifter: Bedava Kaydırma	40
25.2 Adresleme Modları	40
25.3 LDM / STM: Çoklu Yükleme/Saklama	41
26 Koşullu Yürütme ve Bit Komutları	41
26.1 IT Blokları (If-Then)	41
26.2 Bit Alanı ve Sıralama Komutları	41
27 Özel Register'lar, Atomik Erişim ve Bariyerler	42
27.1 Özel Register Erişimi: MRS ve MSR	42
27.2 Atomik Erişim: LDREX ve STREX	42
27.3 Bellek Bariyerleri: DMB, DSB, ISB	43

28 FPU ve Assembly’de Exception İşleme	43
28.1 Kayan Nokta (FPU) Komutları	43
28.2 Assembly ile Exception İşleyici	43
29 Kapanış: Gömülü Geliştirme Yol Haritası	44

KISIM 1

STM32 ve Cortex-M Temelleri

1. Giriş: Mikrodenetleyiciler ve STM32

Mikrodenetleyici (MCU), bir işlemci çekirdeği, bellek (Flash + RAM) ve çevre birimlerini (GPIO, timer, UART. . .) tek bir çip üzerinde toplayan küçük bir bilgisayardır. STM32, STMicroelectronics'in ARM *Cortex-M* çekirdeği kullanan popüler MCU ailesidir; endüstride, robotikte ve hobide yaygındır.

1.1 Mikroişlemci vs Mikrodenetleyici

	Mikroişlemci (PC/telefon)	Mikrodenetleyici (STM32)
Bellek	Harici GB'larca RAM	Dahili KB'larca RAM/Flash
İşletim sistemi	Linux/Windows	Genelde yok (bare-metal) veya RTOS
Çevre birimleri	Harici çipler	Çip içinde tümleşik
Güç	Watt'lar	Miliwatt'lar
Gerçek zaman	Zor (OS gecikmesi)	Deterministik, kolay

1.2 STM32 Ailesi ve Cortex-M Çekirdekleri

STM32 ailesi, ihtiyaca göre farklı Cortex-M çekirdekleri kullanır. Hepsi aynı komut setini (Thumb-2) paylaşır, bu yüzden bilgiler taşınabilir.

Seri	Çekirdek	Kullanım
STM32F0/L0	Cortex-M0/M0+	Düşük maliyet, düşük güç
STM32F1/F3	Cortex-M3/M4	Genel amaçlı (ör. ünlü "Blue Pill")
STM32F4/F7	Cortex-M4/M7	Yüksek performans, FPU, DSP
STM32H7	Cortex-M7	En yüksek performans (480 MHz+)

1.3 Neden Register Seviyesi Öğrenmeli?

Çoğu proje ST'nin HAL (Hardware Abstraction Layer) kütüphanesiyle yazılır — hızlıdır ama neyin nasıl çalıştığını gizler. Bu rehber önce *register seviyesini* (bare-metal) öğretir, çünkü:

- Donanımın gerçekte nasıl çalıştığını anlarsın (datasheet okuma becerisi).
- HAL'ın gizlediği hataları ayıklayabilirsin.
- Kritik yerlerde daha hızlı, daha küçük kod yazabilirsin.
- Assembly'ye (Kısım 5-6) geçiş doğal olur.

Bilgi

Öğrenmek için gerçek bir karta ihtiyacın var. Ucuz ve yaygın seçenekler: **STM32F103 “Blue Pill”** (Cortex-M3, çok ucuz), **STM32 Nucleo** kartları (dahili ST-Link programlayıcı ile) ve **STM32 Discovery** kartları. Programlamak için bir ST-Link programlayıcı/hata ayıklayıcı gerekir (Nucleo'da dahili).

2. Bellek Haritası ve Register Modeli

STM32'yi register seviyesinde programlamak, aslında *belirli bellek adreslerine* değer yazıp okumaktır. Her çevre birimi, bellek haritasında sabit bir adrese “eşlenmiştir” (memory-mapped I/O). Bir GPIO pinini yakmak, o pine karşılık gelen bir register'a bir bit yazmaktan ibarettir.

2.1 Cortex-M Bellek Haritası

Cortex-M, 4 GB'lık düz (flat) bir adres uzayına sahiptir; bu uzay sabit bölgelere ayrılmıştır:

Adres	Bölge	İçerik
0x0000_0000	Code	Flash (program kodu), vektör tablosu
0x2000_0000	SRAM	Değişkenler, yığın (stack), heap
0x4000_0000	Peripheral	Çevre birimi register'ları (GPIO, TIM...)
0xE000_0000	Cortex-M iç	NVIC, SysTick, SCB (sistem register'ları)

2.2 Register'a Erişim: volatile İşaretçiler

Bir çevre birimi register'ına C'de erişmek için, adresini bir volatile işaretçi olarak ele alırız. volatile anahtar kelimesi kritiktir: derleyiciye “bu değer her an donanım tarafından değişebilir, okumaları/yazmaları optimize etme” der.

```

1 #include <stdint.h>
2
3 /* GPIOC çevre biriminin taban adresi (STM32F1 için) */
4 #define GPIOC_BASE 0x40011000UL
5 #define RCC_BASE 0x40021000UL
6
7 /* Register'lara volatile işaretçilerle eriş */
8 #define RCC_APB2ENR (*(volatile uint32_t *) (RCC_BASE + 0x18))
9 #define GPIOC_CRH (*(volatile uint32_t *) (GPIOC_BASE + 0x04))
10 #define GPIOC_ODR (*(volatile uint32_t *) (GPIOC_BASE + 0x0C))
11
12 /* Kullanım: bir register'a doğrudan yaz/oku */
13 /* RCC_APB2ENR |= (1 << 4); // GPIOC saatini aç (birazdan açıklanacak) */

```

Dikkat

volatile'i unutmak, gömülü sistemlerdeki en sinsi hatalardan biridir. Derleyici, “değişmeyen” sandığı bir register okumasını döngüden çıkarabilir veya bir yazmayı tamamen atabilir — kod “bazen çalışır bazen çalışmaz”. Donanım register'larına erişim **her zaman** volatile olmalıdır.

2.3 CMSIS: Standart Register Tanımları

Yukarıdaki gibi adresleri elle tanımlamak yerine, ARM'ın *CMSIS* (Cortex Microcontroller Software Interface Standard) standardı ve ST'nin aygıt başlıkları (stm32f1xx.h) her register'ı hazır, tip güvenli struct'larla tanımlar. Gerçek projede bunları kullanırız.

```
1 #include "stm32f1xx.h"      /* CMSIS aygıt başlığı: tüm register'lar tanımlı */
2
3 /* CMSIS ile aynı işlem çok daha okunaklı: */
4 /* Her çevre birimi bir struct işaretçisidir (GPIOC, RCC, TIM2...) */
5 RCC->APB2ENR |= RCC_APB2ENR_IOPCEN; /* GPIOC saatini aç */
6 GPIOC->ODR   |= (1 << 13);          /* PC13 pinini yüksek yap */
```

Bilgi

CMSIS başlıkları, bir çevre biriminin taban adresini bir struct işaretçisi olarak tanımlar; struct alanları da register'lara karşılık gelir. Örneğin GPIOC->ODR, derleyici tarafından tam olarak *(volatile uint32_t*)(0x40011000 + 0x0C) adresine çevrilir — yani elle yaptığımızla aynı, ama okunaklı ve hatasız.

3. Saat Sistemi ve Araç Zinciri**3.1 Saat (Clock) Sistemi ve RCC**

Güç tasarrufu için STM32'de çevre birimlerinin saatleri *varsayılan olarak kapalıdır*. Bir çevre birimini (GPIO, timer, UART...) kullanmadan önce, onun saatini *RCC* (Reset and Clock Control) üzerinden **açmalısın**. Bu, yeni başlayanların en sık unuttuğu adımdır.

```
1 #include "stm32f1xx.h"
2
3 /* Bir çevre birimini kullanmadan ÖNCE saatini aç: */
4 RCC->APB2ENR |= RCC_APB2ENR_IOPCEN; /* GPIOC (APB2 veri yolunda) */
5 RCC->APB1ENR |= RCC_APB1ENR_TIM2EN; /* TIM2 (APB1 veri yolunda) */
6 RCC->APB2ENR |= RCC_APB2ENR_USART1EN; /* USART1 */
```

Dikkat

“Kodum çalışmıyor” vakalarının bir numaralı sebebi: çevre biriminin saatini açmayı unutmak. Saat kapalıysa register'lara yazdığın değerler *hiçbir etki yapmaz* (ve okumalar 0 döner). Yeni bir çevre birimi kullanırken ilk yapman gereken, doğru RCC register'ında saatini açmaktır — hangi veri yolunda (AHB/APB1/APB2) olduğunu datasheet'ten kontrol et.

3.2 Araç Zinciri ve Derleme

STM32 için kod, arm-none-eabi-gcc çapraz derleyicisiyle (cross-compiler) derlenir — kodu PC'de derleyip MCU için makine kodu üretir.

```
1 # ARM GCC araç zincirini kur (Debian/Ubuntu)
```

```

2 $ sudo apt install gcc-arm-none-eabi gdb-multiarch openocd stlink-tools
3
4 # Derle: startup + linker script gerekir
5 $ arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -Wall -g \
6     -T stm32f103.ld -nostartfiles \
7     startup.s main.c -o firmware.elf
8
9 # ELF'ten ikili (binary) üret
10 $ arm-none-eabi-objcopy -O binary firmware.elf firmware.bin
11
12 # Karta yükle (ST-Link ile)
13 $ st-flash write firmware.bin 0x08000000
14
15 # Boyutu gör (Flash/RAM kullanımı)
16 $ arm-none-eabi-size firmware.elf

```

Bilgi

Bare-metal derleme üç özel parça gerektirir: (1) *startup kodu* (startup.s) — reset sonrası çalışan, yığını kuran ve main'e atlayan assembly; (2) *linker script* (.ld) — kodun Flash'ta, değişkenlerin RAM'de nereye yerleşeceğini tarif eder; (3) *vektör tablosu* — kesme işleyici adreslerini tutar. STM32CubeIDE bunları otomatik üretir; ama nasıl çalıştıklarını bilmek şarttır. Startup kodunu Kısım 6'da assembly ile inceleyeceğiz.

4. Saat Ağacı (Clock Tree) Detayı

STM32'nin ne kadar hızlı çalışacağını ve her çevre biriminin hangi frekansı alacağını *saat ağacı* belirler. Reset sonrası MCU yavaş dahili bir osilatörle çalışır; tam hıza ulaşmak için saat ağacını yapılandırmak gerekir.

4.1 Saat Kaynakları ve PLL

Kaynak	Ad	Özellik
HSI	Dahili yüksek hız	RC osilatör (ör. 8/16 MHz), hazır ama az hassas.
HSE	Harici yüksek hız	Kristal (ör. 8 MHz), hassas.
PLL	Faz kilitli çevrim	Kaynağı çarparak yüksek frekans üretir (ör. 72 MHz).
LSE / LSI	Düşük hız	32.768 kHz — RTC ve watchdog için.

Tipik yol: HSE (8 MHz kristal) → PLL ile $\times 9$ → 72 MHz sistem saati (SYSCLK). Ardından *prescaler*'lar ile AHB, APB1, APB2 veri yolları için türev saatler üretilir.

```

1 #include "stm32f1xx.h"
2
3 /* HSE + PLL ile 72 MHz'e çık (STM32F103) */
4 void clock_init_72mhz(void)
5 {
6     RCC->CR |= RCC_CR_HSEON;                /* harici kristali başlat */
7     while (!(RCC->CR & RCC_CR_HSERDY));      /* hazır olmasını bekle */
8
9     FLASH->ACR |= FLASH_ACR_LATENCY_2;      /* 72 MHz için 2 flash bekleme */

```



```

10
11  /* PLL: kaynak HSE, çarpan x9 -> 8 MHz * 9 = 72 MHz */
12  RCC->CFGR |= RCC_CFGR_PLLSRC | RCC_CFGR_PLLMULL9;
13  RCC->CFGR |= RCC_CFGR_PPRE1_DIV2;          /* APB1 max 36 MHz -> /2 */
14
15  RCC->CR |= RCC_CR_PLLON;                    /* PLL'i başlat */
16  while (!(RCC->CR & RCC_CR_PLLRDY));
17
18  RCC->CFGR |= RCC_CFGR_SW_PLL;                /* sistem saatini PLL yap */
19  while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL);
20 }

```

Dikkat

Saat frekansını artırırken **Flash bekleme durumu** (wait state, FLASH_ACR_LATENCY) doğru ayarlamalısın: Flash belleği CPU kadar hızlı okunamaz, bu yüzden yüksek frekansta ek bekleme gerekir. Ayarı unutmak, MCU'nun kilitlenmesine veya kararsız çalışmasına yol açar. Her frekans aralığı için gereken bekleme sayısı datasheet'te belirtilir.

Bilgi

Saat ağacını elle kurmak yorucudur; ST'nin **STM32CubeMX** aracı, grafik bir saat ağacı editörüyle bu kodu otomatik üretir. Yine de mekanizmayı bilmek, “çevre birimim yanlış frekansta çalışıyor” tarzı hataları çözmek için gereklidir. Timer ve UART frekansları doğrudan APB saatlerine bağlıdır.

KISIM 2

GPIO ve Register Seviyesi Programlama

5. GPIO: Genel Amalı Giriş/ıkış

GPIO (General-Purpose Input/Output) pinleri, MCU'nun dıř d nyayla en temel baėlantısıdır: bir LED yakmak, bir butonu okumak, bir r leyi tetiklemek hep GPIO ile yapılır. Her pin ya *ıkış* (output — MCU pini s rer) ya da *giriş* (input — MCU pini okur) olarak yapılandırılır.

5.1 GPIO Register'ları

Her GPIO portu (GPIOA, GPIOB...) birkaç register'a sahiptir. En  nemlileri:

Register	İřlevi
CRL / CRH	Yapılandırma: pin giriş mi ıkış mı, hız, mod (F1 serisi).
MODER	Pin modu (F0/F4 serisinde CRL/CRH yerine).
IDR	Input Data Register: giriş pinlerinin durumunu <i>okur</i> .
ODR	Output Data Register: ıkış pinlerine deėer <i>yazar</i> .
BSRR	Bit Set/Reset: pinleri atomik olarak 1 veya 0 yapar.

5.2 Bit İşlemleri: Register Programlamanın Dili

Register'larla alıřmak, tek tek *bitleri* ayarlamaktır. Diėer bitleri bozmadan belirli bitleri deėiřtirmek iin bit maskeleri kullanılır. Bu d rt iřlem, g m l  programlamanın temelidir:

İřlem	Kod	Anlamı
Bit kur (set)	$REG \mid= (1 \ll n);$	n . biti 1 yap (diėerlerine dokunma).
Bit temizle (clear)	$REG \&= \sim(1 \ll n);$	n . biti 0 yap.
Bit evir (toggle)	$REG \hat{=} (1 \ll n);$	n . biti tersle.
Bit oku (test)	$(REG \gg n) \& 1$	n . bit 1 mi?

İpucu

“Read-modify-write” kalıbı: $REG \mid= (1 \ll n)$ aslında üç işlemdir — register’ı oku, biti değiştir, geri yaz. Bu yüzden bir kesme araya girerse yarış koşulu olabilir. STM32’nin BSRR register’ı bunu çözer: tek bir yazmayla, atomik olarak pin set/reset yapar — read-modify-write gerekmez.

6. İlk Program: LED Yakma (Blink)

Gömülü dünyanın “merhaba dünya”sı bir LED yakıp söndürmektir. STM32F103 “Blue Pill” kartında dahili LED *PC13* pinine bağlıdır. Adım adım register seviyesinde yapalım.

6.1 Bare-Metal LED Blink

```

1  #include "stm32f1xx.h"
2
3  /* Basit gecikme: işlemciyi meşgul tutan bir döngü (kaba, kalibre değil) */
4  static void delay(volatile uint32_t count)
5  {
6      while (count--) {
7          __asm__("nop"); /* "no operation" -- derleyici döngüyü silmesin */
8      }
9  }
10
11 int main(void)
12 {
13     /* 1) GPIOC saatini aç (RCC üzerinden) */
14     RCC->APB2ENR |= RCC_APB2ENR_IOPCEN;
15
16     /* 2) PC13'ü çıkış olarak yapılandır (F1: CRH, pin 13 -> bit 20-23)
17        MODE = 0b10 (2 MHz çıkış), CNF = 0b00 (push-pull) */
18     GPIOC->CRH &= ~(0xF << 20); /* önce PC13 alanını temizle */
19     GPIOC->CRH |= (0x2 << 20); /* sonra çıkış modunu kur */
20
21     /* 3) Sonsuz döngüde LED'i çevir */
22     while (1) {
23         GPIOC->ODR ^= (1 << 13); /* PC13'ü tersle (toggle) */
24         delay(500000); /* bir süre bekle */
25     }
26 }

```

BSRR ile atomik kontrol

Yukarıda $ODR \hat{=}$... kullandık; ama pini kesin olarak açıp kapamak için BSRR daha güvenlidir. BSRR’nin alt 16 biti pini *set* eder (1 yapar), üst 16 bit *reset* eder (0 yapar): $GPIOC->BSRR = (1 \ll 13)$; pini yakar, $GPIOC->BSRR = (1 \ll (13+16))$; söndürür — her ikisi de tek, atomik yazmadır.

6.2 Aynı İş HAL ile

Karşılaştırma için, aynı işi ST’nin HAL kütüphanesi çok daha yüksek seviyede yapar (ama arkada aynı register’lara yazar):

```

1  #include "stm32f1xx_hal.h"
2
3  int main(void)

```

```

4 {
5     HAL_Init();
6     __HAL_RCC_GPIOC_CLK_ENABLE();          /* saati aç */
7
8     GPIO_InitTypeDef cfg = {0};
9     cfg.Pin   = GPIO_PIN_13;
10    cfg.Mode  = GPIO_MODE_OUTPUT_PP;        /* push-pull çıkış */
11    cfg.Speed = GPIO_SPEED_FREQ_LOW;
12    HAL_GPIO_Init(GPIOC, &cfg);
13
14    while (1) {
15        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
16        HAL_Delay(500);                      /* 500 ms (SysTick tabanlı) */
17    }
18 }

```

Bilgi

Bare-metal vs HAL: Bare-metal kod küçük, hızlı ve şeffaftır ama her register'ı elle ayarlamam gerekir (datasheet okuma). HAL taşınabilir ve hızlı geliştirme sağlar ama daha büyük/yavaştır ve bazen davranışı gizler. Öğrenirken bare-metal, üretimde çoğu zaman HAL (veya LL — Low-Layer) kullanılır. İkisini de anlamak en iyisidir.

7. Giriş Okuma: Buton

Bir butonu okumak, bir GPIO pinini *giriş* olarak yapılandırıp IDR register'ından durumunu okumaktır. Butonun elektriksel olarak tanımlı bir seviyede durması için *pull-up* veya *pull-down* direnci gerekir (STM32'de dahili olarak etkinleştirilebilir).

```

1 #include "stm32f1xx.h"
2
3 int main(void)
4 {
5     /* GPIOA ve GPIOC saatlerini aç */
6     RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPCEN;
7
8     /* PA0'ı giriş + pull-up olarak yapılandır (CRL, pin 0 -> bit 0-3)
9      MODE=0b00 (giriş), CNF=0b10 (pull-up/pull-down) */
10    GPIOA->CRL &= ~(0xF << 0);
11    GPIOA->CRL |= (0x8 << 0);
12    GPIOA->ODR |= (1 << 0);          /* ODR biti pull-UP seçer */
13
14    /* PC13'ü çıkış yap (LED) */
15    GPIOC->CRH &= ~(0xF << 20);
16    GPIOC->CRH |= (0x2 << 20);
17
18    while (1) {
19        /* Butona basılınca PA0 düşer (pull-up olduğu için basılı = 0) */
20        if (((GPIOA->IDR >> 0) & 1) == 0) {
21            GPIOC->BSRR = (1 << 13);    /* LED aç */
22        } else {
23            GPIOC->BSRR = (1 << (13 + 16)); /* LED kapat */
24        }
25    }
26 }

```

Dikkat

Buton sıçraması (debouncing): Mekanik butonlar, basıldığında birkaç milisaniye boyunca hızla açılıp kapanır (bounce). Ham okuma tek bir basışı “çok basış” olarak algılayabilir. Ç z m: bir okuma sonrası kısa bir gecikme ( r. 20 ms) koymak veya durumu birkaç kez  st  ste teyit etmek (software debouncing).

KISIM 3

Çevre Birimleri (Peripherals)

STM32'nin gücü, çip içindeki zengin çevre birimlerinden gelir: zamanlayıcılar, seri iletişim (UART, SPI, I2C), analog-dijital çevirici (ADC) ve daha fazlası. Her biri kendi register'larıyla yapılandırılır. Bu kısım en yaygın olanları tanıtır.

8. Zamanlayıcılar (Timers) ve PWM

Zamanlayıcı (timer), belirli bir saat frekansı ile sayan bir sayaçtır. Kesin zaman aralıkları üretmek, olayları saymak, gecikme oluşturmak ve *PWM* (darbe genişlik modülasyonu) üretmek için kullanılır. PWM, LED parlaklığı ayarlama ve motor/servo kontrolünün temelidir.

8.1 Zamanlayıcı Nasıl Çalışır?

Bir timer, PSC (prescaler — ön bölücü) ile saati böler, sonra ARR (auto-reload register) değerine kadar sayar; oraya ulaştınca sıfırlanıp bir olay (ve isteğe bağlı kesme) üretir. Timer frekansı: $f_{\text{timer}} = \frac{f_{\text{clock}}}{(PSC + 1) \times (ARR + 1)}$.

```

1  #include "stm32f1xx.h"
2
3  /* TIM2'yi 1 Hz periyotla (72 MHz saat varsayımı) yapılandır */
4  void timer_init(void)
5  {
6      RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;    /* TIM2 saatini aç */
7
8      TIM2->PSC = 7200 - 1;                    /* 72 MHz / 7200 = 10 kHz */
9      TIM2->ARR = 10000 - 1;                  /* 10 kHz / 10000 = 1 Hz */
10     TIM2->CR1 |= TIM_CR1_CEN;                /* sayacı başlat (counter enable) */
11 }
12
13 /* Sayacın taşmasını (update event) yoklayarak bekle */
14 void timer_wait_update(void)
15 {
16     while (!(TIM2->SR & TIM_SR_UIF));        /* update flag'i bekle */
17     TIM2->SR &= ~TIM_SR_UIF;                 /* flag'i temizle */
18 }

```

8.2 PWM Üretme

PWM, sabit frekanslı ama değişken *doluluk oranına* (duty cycle) sahip bir kare dalgadır. CCR (capture/compare register) doluluk oranını belirler. Örneğin bir LED'in parlaklığını, göz ortalamayı algıladığı için, PWM ile ayarlarız.

```

1  /* TIM3 Kanal 1'de PWM (PA6 pini) -- LED parlaklık kontrolü */
2  void pwm_init(void)
3  {
4      RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;
5      RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;
6
7      /* PA6'ya alternatif fonksiyon push-pull çıkış yap (CRL bit 24-27) */
8      GPIOA->CRL &= ~(0xF << 24);
9      GPIOA->CRL |= (0xB << 24);          /* MODE=0b11 (50MHz), CNF=0b10 (AF-PP) */
10
11     TIM3->PSC = 72 - 1;                  /* 72 MHz / 72 = 1 MHz sayım */
12     TIM3->ARR = 1000 - 1;                /* 1 MHz / 1000 = 1 kHz PWM */
13     TIM3->CCR1 = 250;                    /* doluluk = 250/1000 = %25 */
14     TIM3->CCMR1 |= (6 << 4);             /* PWM mode 1 (OC1M = 110) */
15     TIM3->CCER |= TIM_CCER_CC1E;        /* kanal 1 çıkışını etkinleştir */
16     TIM3->CR1 |= TIM_CR1_CEN;
17 }
18 /* Parlaklığı değiştirmek için: TIM3->CCR1 = yeni_deger; (0..999) */

```

İpucu

PWM'in doluluk oranını (CCR) çalışırken değiştirerek bir LED'i yavaşça parlatıp söndürebilir (fade), bir servo motorun açısını ayarlayabilir veya bir DC motorun hızını kontrol edebilirsiniz. Frekans (PSC/ARR) sabit kalır, yalnızca CCR değişir.

8.3 HAL Driver ile Timer ve PWM

Periyodik Timer Kesmesi

HAL ile bir timer'ı periyodik kesme üretecek şekilde kurmak birkaç satırdır; her taşmada HAL_TIM_PeriodElapsed çağrılır.

```

1  #include "stm32f1xx_hal.h"
2
3  TIM_HandleTypeDef htim2;
4
5  void timer_hal_init(void)
6  {
7      __HAL_RCC_TIM2_CLK_ENABLE();
8
9      htim2.Instance          = TIM2;
10     htim2.Init.Prescaler     = 7200 - 1; /* 72 MHz / 7200 = 10 kHz */
11     htim2.Init.Period        = 10000 - 1; /* 10 kHz / 10000 = 1 Hz */
12     htim2.Init.CounterMode   = TIM_COUNTERMODE_UP;
13     HAL_TIM_Base_Init(&htim2);
14
15     HAL_TIM_Base_Start_IT(&htim2);      /* kesmeli olarak başlat */
16     HAL_NVIC_EnableIRQ(TIM2_IRQn);
17 }
18
19 /* Her periyot dolduğunda (1 Hz) çağrılır */
20 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
21 {
22     if (htim->Instance == TIM2) {
23         HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13); /* LED çevir */
24     }
25 }

```

```

26
27 void TIM2_IRQHandler(void) { HAL_TIM_IRQHandler(&htim2); }

```

HAL ile PWM

```

1 TIM_HandleTypeDef htim3;
2
3 void pwm_hal_init(void)
4 {
5     __HAL_RCC_TIM3_CLK_ENABLE();
6
7     htim3.Instance      = TIM3;
8     htim3.Init.Prescaler = 72 - 1;      /* 1 MHz sayım */
9     htim3.Init.Period    = 1000 - 1;    /* 1 kHz PWM */
10    HAL_TIM_PWM_Init(&htim3);
11
12    TIM_OC_InitTypeDef oc = {0};
13    oc.OCMode      = TIM_OCMode_PWM1;
14    oc.Pulse       = 250;                /* doluluk = %25 (250/1000) */
15    oc.OCpolarity  = TIM_OCPolarity_HIGH;
16    HAL_TIM_PWM_ConfigChannel(&htim3, &oc, TIM_CHANNEL_1);
17
18    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1); /* PWM çıkışını başlat */
19 }
20
21 /* Çalışırken doluluk oranını değiştir (ör. LED parlaklığı / motor hızı) */
22 void pwm_set_duty(uint16_t duty)          /* 0..999 */
23 {
24     __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, duty);
25 }

```

Input Capture: Sinyal Ölçme

Timer'ın bir başka gücü *input capture*'dır: bir giriş pinindeki kenar geldiğinde sayaç değerini yakalar. Böylece bir sinyalin frekansını veya darbe genişliğini (ör. HC-SR04 mesafe sensörü, RC alıcı) ölçebilirsiniz.

```

1 /* Kenar yakalandığında çağrılır -- iki yakalama arası fark periyodu verir */
2 void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
3 {
4     if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1) {
5         uint32_t captured = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
6         /* Ardışık iki 'captured' farkı -> periyot -> frekans hesaplanır */
7     }
8 }
9 /* Başlatma: HAL_TIM_IC_Start_IT(&htim, TIM_CHANNEL_1); */

```

Encoder modu

Timer'lar ayrıca *encoder modunda* çalışabilir: bir rotary encoder'ın (kadran/motor mili) iki fazını (A, B) sayarak konum ve yön bilgisini donanımda üretir. HAL'da `HAL_TIM_Encoder_Start()` ile başlatılır ve sayaç değeri (`__HAL_TIM_GET_COUNTER`) doğrudan pozisyonu verir — CPU hiç yorulmaz.

9. Seri İletişim: UART

UART (Universal Asynchronous Receiver/Transmitter), iki cihaz arasında iki tel (TX, RX) üzerinden seri veri gönderen en yaygın iletişim yöntemidir. PC ile hata ayıklama mesajı alışverişinde (printf tarzı) vazgeçilmezdir.

9.1 UART Yapılandırma ve Gönderme

UART'ın en önemli ayarı *baud rate*'tir (saniyedeki bit sayısı, ör. 9600 veya 115200) — iki taraf aynı olmalıdır. Baud, BRR register'ıyla ayarlanır.

```

1  #include "stm32f1xx.h"
2
3  void uart_init(void)
4  {
5      RCC->APB2ENR |= RCC_APB2ENR_USART1EN | RCC_APB2ENR_IOPAEN;
6
7      /* PA9 = TX (alternatif fonksiyon push-pull) */
8      GPIOA->CRH &= ~(0xF << 4);
9      GPIOA->CRH |= (0xB << 4);
10
11     /* Baud = 115200 @ 72 MHz -> BRR = 72e6 / 115200 = 625 */
12     USART1->BRR = 625;
13     USART1->CR1 |= USART_CR1_TE;          /* verici (transmitter) etkin */
14     USART1->CR1 |= USART_CR1_UE;         /* USART'ı etkinleştir */
15 }
16
17 /* Tek bir karakter gönder (gönderme tamponu boşalana kadar bekle) */
18 void uart_send_char(char c)
19 {
20     while (!(USART1->SR & USART_SR_TXE)); /* TXE: tampon boş mu? */
21     USART1->DR = c;                       /* veriyi yaz -> otomatik gönderilir */
22 }
23
24 /* Bir string gönder */
25 void uart_send_string(const char *s)
26 {
27     while (*s) {
28         uart_send_char(*s++);
29     }
30 }

```

Bilgi

UART ile PC'ye mesaj göndermek, gömülü hata ayıklamanın en pratik yoludur (bir tür “printf debugging”). Bir USB-Serial dönüştürücü (FTDI/CP2102) ile MCU'nun TX pinini PC'ye bağlar, PC'de bir terminal (minicom, PuTTY, screen) açarsın. printf'i UART'a yönlendirmek için `_write` sistem çağrısını yeniden tanımlayabilirsin.

9.2 HAL Driver ile UART

Gerçek projelerde UART'ı register seviyesinde yönetmek yerine ST'nin *HAL* (Hardware Abstraction Layer) sürücüsü kullanılır. HAL, tüm register detaylarını gizler; sadece bir *handle* yapısını (UART_HandleTypeDef) doldurup hazır fonksiyonları çağırırsın. Aşağıda üç çalışma modu gösterilir: *blocking* (yoklamak), *interrupt* ve *DMA*.

Yapılandırma (Init)

```

1  #include "stm32f1xx_hal.h"
2
3  UART_HandleTypeDef huart1;          /* UART handle (durum + ayarlar) */
4
5  void uart_hal_init(void)
6  {
7      __HAL_RCC_USART1_CLK_ENABLE();    /* saatleri aç */
8      __HAL_RCC_GPIOA_CLK_ENABLE();
9
10     /* PA9=TX, PA10=RX pinlerini alternatif fonksiyon olarak ayarla */
11     GPIO_InitTypeDef gpio = {0};
12     gpio.Pin   = GPIO_PIN_9;
13     gpio.Mode  = GPIO_MODE_AF_PP;      /* alternatif fonksiyon push-pull */
14     gpio.Speed = GPIO_SPEED_FREQ_HIGH;
15     HAL_GPIO_Init(GPIOA, &gpio);
16     gpio.Pin   = GPIO_PIN_10;
17     gpio.Mode  = GPIO_MODE_INPUT;      /* RX girişi */
18     HAL_GPIO_Init(GPIOA, &gpio);
19
20     /* UART parametreleri */
21     huart1.Instance = USART1;
22     huart1.Init.BaudRate = 115200;     /* baud hızı */
23     huart1.Init.WordLength = UART_WORDLENGTH_8B;
24     huart1.Init.StopBits = UART_STOPBITS_1;
25     huart1.Init.Parity = UART_PARITY_NONE;
26     huart1.Init.Mode = UART_MODE_TX_RX; /* hem gönder hem al */
27     huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
28     HAL_UART_Init(&huart1);           /* tüm register'ları HAL ayarlar */
29 }

```

Blocking (Yoklamalı) Gönderme ve Alma

En basit mod: fonksiyon, işlem bitene (veya zaman aşımına) kadar bekler.

```

1  uint8_t rx_byte;
2  char msg[] = "Merhaba UART\r\n";
3
4  /* Gönder: işlem bitene kadar bekle (son parametre = zaman aşımı ms) */
5  HAL_UART_Transmit(&huart1, (uint8_t *)msg, sizeof(msg) - 1, HAL_MAX_DELAY);
6
7  /* Al: 1 bayt gelene kadar bekle */
8  HAL_UART_Receive(&huart1, &rx_byte, 1, HAL_MAX_DELAY);

```

Interrupt Modu (Non-Blocking)

Interrupt modunda CPU beklemes; veri gelince/gidince HAL bir *callback* çağırır. Bu, işlemciyi bloklamadan arka planda iletişim sağlar.

```

1  uint8_t rx_buffer[1];
2
3  void uart_start_receive(void)
4  {
5      /* 1 bayt için kesmeli alım başlat -- hemen döner, beklemes */
6      HAL_UART_Receive_IT(&huart1, rx_buffer, 1);
7  }
8
9  /* Alım tamamlanınca HAL bunu otomatik çağırır (callback'i sen yazarsın) */
10 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
11 {

```

```

12     if (huart->Instance == USART1) {
13         /* Gelen baytı işle (örneğin geri gönder - echo) */
14         HAL_UART_Transmit_IT(&huart1, rx_buffer, 1);
15         HAL_UART_Receive_IT(&huart1, rx_buffer, 1); /* tekrar dinle */
16     }
17 }
18
19 /* USART1 kesmesi HAL'ın işleyicisine yönlendirilmeli (ISR): */
20 void USART1_IRQHandler(void)
21 {
22     HAL_UART_IRQHandler(&huart1); /* HAL kesmeyi çözer, callback'i çağırır */
23 }

```

DMA Modu (Yüksek Verim)

Büyük veri bloklarını CPU'yu hiç meşgul etmeden aktarmak için DMA kullanılır.

```

1 /* Bir bloğu DMA ile gönder -- CPU başka iş yaparken transfer arka planda olur */
2 HAL_UART_Transmit_DMA(&huart1, (uint8_t *)msg, sizeof(msg) - 1);
3
4 /* DMA transferi bitince çağrılır */
5 void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
6 {
7     /* Gönderim tamamlandı -- bir sonraki bloğu başlatabilirsin */
8 }

```

printf'i UART'a Yönlendirme

printf'in UART'a çıkmasını sağlamak için `_write` (veya `__io_putchar`) sistem fonksiyonunu HAL üzerinden yeniden tanımlarız:

```

1 #include <stdio.h>
2
3 /* newlib'in printf'i sonunda buraya gelir -> UART'a yaz */
4 int _write(int file, char *ptr, int len)
5 {
6     HAL_UART_Transmit(&huart1, (uint8_t *)ptr, len, HAL_MAX_DELAY);
7     return len;
8 }
9 /* Artık: printf("Sıcaklık: %d C\n", temp); -> UART terminaline çıkar */

```

Bilgi

Hangi mod ne zaman? *Blocking* en basittir, kısa mesajlar için idealdir ama CPU'yu bekletir. *Interrupt* modu, arka planda sürekli komut/veri alışverişi (ör. bir konsol) için uygundur. *DMA*, yüksek hızlı veya büyük bloklar (ör. sensör akışı, log) için CPU'yu tamamen serbest bırakır. HAL'ın güzelliği, aynı sürücüyle üç modu da kolayca kullanabilmendir.

10. ADC ve Diğer Veri Yolları (SPI, I2C)

10.1 ADC: Analog Okuma

ADC (Analog-to-Digital Converter), bir pindeki analog gerilimi (0–3.3 V) bir sayıya çevirir. Potansiyometre, sıcaklık sensörü, ışık sensörü gibi analog kaynakları okumak için kullanılır. STM32'nin 12-bit ADC'si 0–4095 arası değer üretir.

```

1  #include "stm32f1xx.h"
2
3  void adc_init(void)
4  {
5      RCC->APB2ENR |= RCC_APB2ENR_ADC1EN | RCC_APB2ENR_IOPAEN;
6      /* PA0 varsayılan olarak analog giriş yapabilir (CRL sıfırla) */
7      GPIOA->CRL &= ~(0xF << 0);          /* PA0 = analog mod (0b0000) */
8
9      ADC1->CR2 |= ADC_CR2_ADON;          /* ADC'yi aç */
10 }
11
12 /* Kanal 0'dan (PA0) bir örnek oku */
13 uint16_t adc_read(void)
14 {
15     ADC1->SQR3 = 0;                      /* dönüşüm sırası: kanal 0 */
16     ADC1->CR2 |= ADC_CR2_ADON;          /* dönüşümü başlat */
17     while (!(ADC1->SR & ADC_SR_EOC));    /* dönüşüm bitti mi? (EOC) */
18     return ADC1->DR;                    /* 12-bit sonucu oku (0..4095) */
19 }
20
21 /* Değeri gerilime çevir: voltage = adc_read() * 3.3 / 4095 */

```

10.2 HAL Driver ile ADC

Aynı ADC okumasını HAL sürücüsüyle yapmak hem daha okunaklı hem de kesme/DMA modlarına kolayca geçilebilir. Bir ADC_HandleTypeDef handle'ı yapılandırıp hazır fonksiyonları çağırırız.

Yapılandırma ve Yoklamalı (Polling) Okuma

```

1  #include "stm32f1xx_hal.h"
2
3  ADC_HandleTypeDef hadc1;                /* ADC handle */
4
5  void adc_hal_init(void)
6  {
7      __HAL_RCC_ADC1_CLK_ENABLE();
8      __HAL_RCC_GPIOA_CLK_ENABLE();
9
10     GPIO_InitTypeDef gpio = {0};
11     gpio.Pin = GPIO_PIN_0;
12     gpio.Mode = GPIO_MODE_ANALOG;        /* PA0 = analog giriş */
13     HAL_GPIO_Init(GPIOA, &gpio);
14
15     hadc1.Instance = ADC1;
16     hadc1.Init.ContinuousConvMode = DISABLE; /* tek seferlik dönüşüm */
17     hadc1.Init.ScanConvMode = DISABLE;
18     hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
19     hadc1.Init.NbrOfConversion = 1;
20     HAL_ADC_Init(&hadc1);
21
22     /* Kanal 0'ı (PA0) dönüşüm sırasına ekle */
23     ADC_ChannelConfTypeDef ch = {0};
24     ch.Channel = ADC_CHANNEL_0;
25     ch.Rank = 1;
26     ch.SamplingTime = ADC_SAMPLETIME_55CYCLES_5;
27     HAL_ADC_ConfigChannel(&hadc1, &ch);
28 }
29

```

```

30  /* Yoklamalı okuma: dönüşümü başlat, bitmesini bekle, değeri al */
31  uint16_t adc_hal_read(void)
32  {
33      HAL_ADC_Start(&hadc1);
34      HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY); /* dönüşümü bekle */
35      uint16_t value = HAL_ADC_GetValue(&hadc1); /* 12-bit sonuç */
36      HAL_ADC_Stop(&hadc1);
37      return value;
38  }

```

Interrupt Modu

```

1  volatile uint16_t adc_value;
2
3  void adc_start_it(void)
4  {
5      HAL_ADC_Start_IT(&hadc1); /* kesmeli dönüşüm başlat, beklemez */
6  }
7
8  /* Dönüşüm bitince HAL bunu otomatik çağırır */
9  void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
10 {
11     adc_value = HAL_ADC_GetValue(hadc); /* sonucu al */
12 }
13
14 void ADC1_2_IRQHandler(void)
15 {
16     HAL_ADC_IRQHandler(&hadc1);
17 }

```

DMA ile Sürekli Çok-Kanal Okuma

Birden çok analog kanalı sürekli ve CPU'suz okumanın en verimli yolu DMA'dır. HAL, DMA'yı ADC'ye bağlayıp bir tampona otomatik yazar.

```

1  uint16_t adc_buffer[3]; /* 3 kanal için tampon */
2
3  /* ScanConvMode=ENABLE, ContinuousConvMode=ENABLE, NbrOfConversion=3 ile
4  init edilmiş varsayılır; 3 kanal rank 1,2,3'e atanır. */
5  void adc_start_dma(void)
6  {
7      /* Dönüşümleri başlat; her sonuç otomatik adc_buffer[]'a yazılır */
8      HAL_ADC_Start_DMA(&hadc1, (uint32_t *)adc_buffer, 3);
9  }
10 /* Artık adc_buffer[0..2] her zaman en güncel 3 kanal değerini tutar -- CPU serbest */

```

İpucu

DMA + *scan mode* kombinasyonu, birden çok sensörü (ör. 3 eksenli potansiyometre veya çoklu sıcaklık) tek ADC ile sürekli okumanın standart yoludur. Değerler arka planda tampona akar; ana kod sadece tampondan en güncel değeri okur. Bu kalıp gömülü sistemlerde çok yaygındır.

10.3 SPI ve I2C: Sensör Veri Yolları

Çoğu sensör ve harici çip (ekran, EEPROM, IMU) MCU ile *SPI* veya *I2C* veri yolu üzerinden konuşur. İkisi de senkron seri protokoldür ama farklı ödünleşmeleri vardır.

Özellik	SPI	I2C
Tel sayısı	4 (MOSI, MISO, SCK, CS)	2 (SDA, SCL)
Hız	Çok hızlı (10+ MHz)	Orta (100k–400k–1M)
Cihaz sayısı	Her cihaza ayrı CS	Adresle çok cihaz (aynı 2 tel)
Karmaşıklık	Basit	Adresleme + ACK protokolü
Kullanım	Ekran, SD kart, hızlı ADC	Sensör, EEPROM, RTC

```

1  /* SPI ile tek bayt gönder/al (full-duplex: aynı anda hem gönderir hem alır) */
2  uint8_t spi_transfer(uint8_t data)
3  {
4      while (!(SPI1->SR & SPI_SR_TXE));    /* gönderme tamponu boş mu? */
5      SPI1->DR = data;                      /* veriyi gönder */
6      while (!(SPI1->SR & SPI_SR_RXNE));    /* yanıt geldi mi? */
7      return SPI1->DR;                     /* gelen baytı oku */
8  }

```

HAL ile SPI

HAL, SPI protokolünün tüm zamanlama detaylarını gizler. Bir SPI_HandleTypeDef yapılandırıp gönder/al fonksiyonlarını çağırırsın.

```

1  #include "stm32f1xx_hal.h"
2
3  SPI_HandleTypeDef hspi1;
4
5  void spi_hal_init(void)
6  {
7      __HAL_RCC_SPI1_CLK_ENABLE();
8      /* ... GPIO: SCK, MOSI, MISO pinlerini AF olarak ayarla ... */
9
10     hspi1.Instance          = SPI1;
11     hspi1.Init.Mode         = SPI_MODE_MASTER;    /* ana (master) cihaz */
12     hspi1.Init.Direction    = SPI_DIRECTION_2LINES; /* full-duplex */
13     hspi1.Init.DataSize     = SPI_DATASIZE_8BIT;
14     hspi1.Init.CLKPolarity  = SPI_POLARITY_LOW;   /* SPI mode 0 */
15     hspi1.Init.CLKPhase     = SPI_PHASE_1EDGE;
16     hspi1.Init.NSS          = SPI_NSS_SOFT;      /* CS'i yazılım yönetir */
17     hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_8;
18     HAL_SPI_Init(&hspi1);
19 }
20
21 void spi_hal_example(void)
22 {
23     uint8_t tx[2] = {0x9F, 0x00};    /* gönderilecek komut */
24     uint8_t rx[2];
25
26     /* CS'i indir (bir GPIO pini) -> aktarım -> CS'i kaldır */
27     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET);
28     HAL_SPI_TransmitReceive(&hspi1, tx, rx, 2, HAL_MAX_DELAY); /* aynı anda gönder+al */
29     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);
30     /* Sadece göndermek için: HAL_SPI_Transmit(&hspi1, tx, 2, HAL_MAX_DELAY); */
31 }

```

HAL ile I2C

I2C’de her cihazın bir *adres*i vardır. HAL, adresleme + ACK protokolünü otomatik yönetir. Özellikle *register okuma/yazma* için Mem_Read/ Mem_Write fonksiyonları çok pratiktir (sensörlerin çoğu register tabanlıdır).

```

1 I2C_HandleTypeDef hi2c1;
2
3 void i2c_hal_init(void)
4 {
5     __HAL_RCC_I2C1_CLK_ENABLE();
6     /* ... GPIO: SCL, SDA pinlerini AF open-drain olarak ayarla ... */
7
8     hi2c1.Instance      = I2C1;
9     hi2c1.Init.ClockSpeed = 100000;      /* 100 kHz standart mod */
10    hi2c1.Init.DutyCycle  = I2C_DUTYCYCLE_2;
11    hi2c1.Init.OwnAddress1 = 0;
12    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
13    HAL_I2C_Init(&hi2c1);
14 }
15
16 void i2c_hal_example(void)
17 {
18     uint8_t dev_addr = 0x68 << 1;      /* 7-bit adres, HAL 8-bit ister */
19     uint8_t data;
20
21     /* Bir sensör register'ından oku (ör. MPU6050 WHO_AM_I = reg 0x75) */
22     HAL_I2C_Mem_Read(&hi2c1, dev_addr, 0x75, I2C_MEMADD_SIZE_8BIT,
23                     &data, 1, HAL_MAX_DELAY);
24
25     /* Bir register'a yaz */
26     uint8_t config = 0x00;
27     HAL_I2C_Mem_Write(&hi2c1, dev_addr, 0x6B, I2C_MEMADD_SIZE_8BIT,
28                      &config, 1, HAL_MAX_DELAY);
29 }

```

Bilgi

SPI ve I2C’yi register seviyesinde elle yönetmek zahmetlidir (zamanlama, protokol detayları). Bu iki veri yolu, HAL kütüphanesinin gerçekten değer kattığı yerlerdir. Her ikisinin de kesme (_IT) ve DMA (_DMA) sürümleri vardır: örneğin HAL_SPI_Transmit_DMA() ile bir ekranı CPU’yu meşgul etmeden güncelleyebilirsiniz. Register seviyesini anlamak yine de hata ayıklama için faydalıdır.

KISIM 4

Kesmeler, NVIC ve DMA

11. Kesmeler ve NVIC

Şimdiye kadarki örneklerde donanımı sürekli *yokladık* (polling — “veri geldi mi? geldi mi?”). Bu, işlemciyi meşgul eder ve verimsizdir. *Kesme* (interrupt), donanımın “bir olay oldu!” diye işlemciyi *uyandırmasıdır*: işlemci ne yapıyorsa duraklatır, olaya özel bir işleyici (ISR) çalıştırır, sonra kaldığı yerden devam eder.

11.1 NVIC: Kesme Denetleyicisi

Cortex-M’de kesmeleri *NVIC* (Nested Vectored Interrupt Controller) yönetir. NVIC, hangi kesmelerin etkin olduğunu, önceliklerini ve iç içe (nested) çalışmasını denetler. Her kesme kaynağının bir numarası (IRQ number) ve bir işleyici adresi (vektör tablosunda) vardır.

Kavram	Açıklama
Vektör tablosu	Her kesme için işleyici fonksiyon adresini tutan tablo (Flash başında).
IRQ numarası	Her kesme kaynağının kimliği (ör. TIM2_IRQn).
Öncelik (priority)	Küçük sayı = yüksek öncelik; yüksek olan düşüğü kesebilir.
ISR	Interrupt Service Routine — kesme gelince çalışan fonksiyon.

11.2 Timer Kesmesi Örneği

Bir timer’ın periyodik olarak kesme üretmesini sağlayalım — böylece `delay` ile beklemek yerine, işlemci başka iş yaparken timer arka planda LED’i çevirir.

```

1  #include "stm32f1xx.h"
2
3  void timer_irq_init(void)
4  {
5      RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
6
7      TIM2->PSC = 7200 - 1;           /* 10 kHz */
8      TIM2->ARR = 10000 - 1;         /* 1 Hz taşma */
9      TIM2->DIER |= TIM_DIER_UIE;    /* update kesmesini etkinleştir */
10     TIM2->CR1 |= TIM_CR1_CEN;
11
12     NVIC_SetPriority(TIM2_IRQn, 2); /* öncelik ata */

```



```

13     NVIC_EnableIRQ(TIM2_IRQn);           /* NVIC'te bu kesmeyi aç */
14 }
15
16 /* ISR: adı vektör tablosundaki isimle TAM eşleşmeli (startup dosyasından) */
17 void TIM2_IRQHandler(void)
18 {
19     if (TIM2->SR & TIM_SR_UIF) {          /* kesme kaynağını doğrula */
20         TIM2->SR &= ~TIM_SR_UIF;          /* bayrağı temizle (ŞART!) */
21         GPIOC->ODR ^= (1 << 13);         /* LED'i çevir */
22     }
23 }

```

Dikkat

ISR'de kesme bayrağını temizlemeyi unutma! Bayrağı temizlemezsen, işleyiciden çıkar çıkmaz kesme *hemen tekrar* tetiklenir — program sonsuza dek ISR'de kalır (kilitlenme gibi görünür). Ayrıca ISR'lerin *kısa* olması ve içinde HAL_Delay gibi bloklayan çağrılar bulunmaması gerekir. ISR adının startup dosyasındaki vektör adıyla birebir aynı olması da şarttır.

11.3 Harici Kesme (EXTI): Buton ile

Bir GPIO pinindeki değişimi (butona basılması) yoklamadan yakalamak için *EXTI* (External Interrupt) kullanılır. Pin durumu değiştiğinde otomatik kesme üretilir.

```

1 void exti_button_init(void)
2 {
3     RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_AFIOEN;
4     /* PA0'ı giriş yap (CRL) ... (giriş yapılandırması yukarıdaki gibi) */
5
6     /* EXTI hattı 0'ı PA0'a bağla (AFIO üzerinden) */
7     AFIO->EXTICR[0] &= ~AFIO_EXTICR1_EXTIO; /* port A seç */
8     EXTI->IMR |= EXTI_IMR_MR0;              /* hat 0 kesmesini aç */
9     EXTI->FTSR |= EXTI_FTSR_TR0;            /* düşen kenarda tetikle */
10
11     NVIC_EnableIRQ(EXTIO_IRQn);
12 }
13
14 void EXTI_IRQHandler(void)
15 {
16     if (EXTI->PR & EXTI_PR_PRO) {           /* hat 0 mı tetikledi? */
17         EXTI->PR = EXTI_PR_PRO;             /* pending bit'i temizle (1 yazarak) */
18         GPIOC->ODR ^= (1 << 13);           /* butona basılınca LED çevir */
19     }
20 }

```

11.4 HAL Driver ile Harici Kesme

HAL ile EXTI kurmak çok daha kısadır: pini GPIO_MODE_IT_FALLING moduyla iklendirir, NVIC'i açar ve ortak HAL_GPIO_EXTI_Callback fonksiyonunu yazarsın.

```

1 #include "stm32f1xx_hal.h"
2
3 void exti_hal_init(void)
4 {
5     __HAL_RCC_GPIOA_CLK_ENABLE();
6
7     GPIO_InitTypeDef gpio = {0};

```

```

8   gpio.Pin = GPIO_PIN_0;
9   gpio.Mode = GPIO_MODE_IT_FALLING;      /* düşen kenarda kesme */
10  gpio.Pull = GPIO_PULLUP;
11  HAL_GPIO_Init(GPIOA, &gpio);
12
13  HAL_NVIC_SetPriority(EXTIO_IRQn, 2, 0);
14  HAL_NVIC_EnableIRQ(EXTIO_IRQn);
15 }
16
17 /* ISR sadece HAL'a yönlendirir */
18 void EXTI_IRQHandler(void) { HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0); }
19
20 /* HAL, doğru pini bulup bu ortak callback'i çağırır */
21 void HAL_GPIO_EXTI_Callback(uint16_t pin)
22 {
23     if (pin == GPIO_PIN_0) {
24         HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
25     }
26 }

```

12. SysTick ve Zamanlama

SysTick, her Cortex-M çekirdeğinde bulunan basit bir 24-bit geri sayım zamanlayıcısıdır. Genelde işletim sistemleri veya HAL_Delay için periyodik bir “sistem tik”i (ör. her 1 ms) üretmek üzere kullanılır — donanımdan bağımsız olduğu için taşınabilir.

```

1  #include "stm32f1xx.h"
2
3  volatile uint32_t ms_ticks = 0;          /* geçen milisaniye sayacı */
4
5  void systick_init(void)
6  {
7      /* Her 1 ms'de bir kesme: 72 MHz / 1000 = 72000 sayım */
8      SysTick_Config(72000);               /* CMSIS: yükle + kesmeyi aç */
9  }
10
11 /* SysTick ISR'si: her 1 ms'de bir çalışır */
12 void SysTick_Handler(void)
13 {
14     ms_ticks++;
15 }
16
17 /* Milisaniye cinsinden bloklamalı gecikme */
18 void delay_ms(uint32_t ms)
19 {
20     uint32_t start = ms_ticks;
21     while ((ms_ticks - start) < ms);      /* hedef süre geçene kadar bekle */
22 }

```

Bilgi

SysTick tabanlı delay_ms, meşgul döngüyle (nop sayan) gecikmeden çok daha doğrudur: kesin milisaniye verir ve derleyici optimizasyonundan etkilenmez. ms_ticks'in volatile olması kritiktir — çünkü ISR'de değişir, ana kod ise onu okur. volatile olmazsa derleyici delay_ms döngüsünü sonsuza kadar sürdürebilir.

13. DMA: İşlemciyi Meşgul Etmeden Veri Taşıma

DMA (Direct Memory Access), bellek ile çevre birimleri (veya bellekten belleğe) arasında veriyi *işlemci olmadan* taşıyan bir birimdir. Örneğin ADC'den 1000 örneği okuyup RAM'e yazmak; DMA olmadan işlemci her örneği tek tek kopyalar (yavaş, meşgul), DMA ile ise işlemci başka iş yaparken transfer arka planda yapılır.

DMA olmadan

DMA ile

İşlemci her baytı elle kopyalar Donanım kopyalar; işlemci serbest

Yüksek CPU yükü Neredeyse sıfır CPU yükü

Yüksek hızda veri kaybı riski Hızlı, sürekli akış (ADC, UART, SPI)

```

1  /* ADC'den belleğe DMA ile sürekli okuma (kavramsal iskelet) */
2  void adc_dma_init(uint16_t *buffer, uint16_t length)
3  {
4      RCC->AHBENR |= RCC_AHBENR_DMA1EN;      /* DMA1 saatini aç */
5
6      DMA1_Channel1->CPAR = (uint32_t)&ADC1->DR; /* kaynak: ADC veri register'ı */
7      DMA1_Channel1->CMAR = (uint32_t)buffer;   /* hedef: RAM tamponu */
8      DMA1_Channel1->CNDTR = length;           /* aktarılabilecek eleman sayısı */
9      DMA1_Channel1->CCR = DMA_CCR_MINC |      /* bellek adresini artır */
10                          DMA_CCR_CIRC |      /* dairesel mod (sürekli) */
11                          DMA_CCR_EN;        /* DMA kanalını başlat */
12      /* Artık ADC her dönüşümü otomatik olarak buffer[]'a yazar -- CPU serbest */
13 }

```

İpucu

DMA, gömülü sistemlerde yüksek verimin anahtarıdır. Sürekli ADC örnekleme, hızlı UART/SPI akışı, ekran güncelleme gibi işlerde işlemciyi tamamen boşa çıkarır. HAL'da HAL_ADC_Start_DMA() gibi fonksiyonlar DMA kurulumunu kolaylaştırır. Dairesel (circular) mod, aynı tamponu sürekli döngüde doldurarak kesintisiz akış sağlar.

KISIM 5

Sistem Kaynakları: RTC, Watchdog ve Düşük Güç (HAL)

STM32, çevresel iletişimin ötesinde sistem düzeyinde birçok kaynak sunar: gerçek zaman saati (RTC), sistemi kilitlenmelerden koruyan watchdog ve pil ömrünü uzatan düşük güç modları. Bunları HAL sürücülerıyla inceliyoruz.

14. RTC: Gerçek Zaman Saati

RTC (Real-Time Clock), tarih ve saati (yıl/ay/gün, saat/dakika/saniye) tutan bir çevre birimidir. Ayrı bir düşük hızlı kristalle (LSE, 32.768 kHz) çalışır ve ana güç kesilse bile bir pil (VBAT) ile saymaya devam edebilir. Veri kaydedicilerde ve zamanlama gerektiren uygulamalarda temeldir.

```

1  #include "stm32f1xx_hal.h"
2
3  RTC_HandleTypeDef hrtc;
4
5  void rtc_hal_init(void)
6  {
7      hrtc.Instance = RTC;
8      hrtc.Init.AsynchPrediv = RTC_AUTO_1_SECOND;  /* 1 saniyelik tik */
9      HAL_RTC_Init(&hrtc);
10 }
11
12 void rtc_set_time(void)
13 {
14     RTC_TimeTypeDef t = {0};
15     t.Hours = 14; t.Minutes = 30; t.Seconds = 0;
16     HAL_RTC_SetTime(&hrtc, &t, RTC_FORMAT_BIN);
17
18     RTC_DateTypeDef d = {0};
19     d.Year = 26; d.Month = 7; d.Date = 13;  /* 13 Temmuz 2026 */
20     HAL_RTC_SetDate(&hrtc, &d, RTC_FORMAT_BIN);
21 }
22
23 void rtc_read_time(void)
24 {
25     RTC_TimeTypeDef t;
26     RTC_DateTypeDef d;
27     HAL_RTC_GetTime(&hrtc, &t, RTC_FORMAT_BIN);
28     HAL_RTC_GetDate(&hrtc, &d, RTC_FORMAT_BIN);  /* GetTime'dan SONRA çağır! */
29     printf("%02d:%02d:%02d\n", t.Hours, t.Minutes, t.Seconds);
30 }

```

Dikkat

RTC’de HAL_RTC_GetTime’den sonra **mutlaka** HAL_RTC_GetDate çağır — saati okumak, tarih register’ını kilitler ve tarihi okuyana kadar güncellenmez. Sırayı bozarsan tarih/saat tutarsız okunur. Ayrıca RTC’yi yeniden başlatmamak için ilk kurulumda backup register’a bir imza yazıp kontrol etmek yaygın bir kalıptır.

15. Watchdog: Sistemi Kilitlenmeden Koruma

Watchdog (bekçi köpeği), yazılım kilitlenirse (sonsuz döngü, çökme) sistemi otomatik *yeniden başlatan* bir zamanlayıcıdır. Fikir: kod düzenli aralıklarla watchdog’u “besler” (reset’ler); beslemeyi durdurursa (kilitlendi demektir) watchdog süre dolunca MCU’yu resetler. Güvenlik kritik gömülü sistemlerde şarttır.

```

1  #include "stm32f1xx_hal.h"
2
3  IWDG_HandleTypeDef hiwdg;
4
5  void watchdog_hal_init(void)
6  {
7      hiwdg.Instance      = IWDG;
8      hiwdg.Init.Prescaler = IWDG_PRESCALER_64;
9      hiwdg.Init.Reload   = 1250;           /* ~2 saniye zaman aşımı (40kHz LSI) */
10     HAL_IWDG_Init(&hiwdg);              /* başlar başlamaz saymaya başlar */
11 }
12
13 int main(void)
14 {
15     HAL_Init();
16     watchdog_hal_init();
17
18     while (1) {
19         do_work();                       /* normal iş */
20         HAL_IWDG_Refresh(&hiwdg);        /* watchdog’u besle (reset’i geciktir) */
21         /* Eğer do_work() kilitlenirse Refresh çağrılmaz -> MCU resetlenir */
22     }
23 }

```

Bilgi

İki tür watchdog vardır: **IWDG** (Independent Watchdog) bağımsız düşük hızlı saatle çalışır — ana saat dursa bile korur, en sağlamı. **WWDG** (Window Watchdog) ise beslemenin belirli bir “pencere” içinde yapılmasını ister — hem çok geç hem çok erken beslemeyi hata sayar, daha katı zamanlama denetimi sağlar. Kritik uygulamalarda IWDG standarttır.

16. Düşük Güç Modları

Pil ile çalışan cihazlarda (IoT sensörü, giyilebilir) güç tüketimi kritiktir. STM32, işi olmadığında uykuya geçerek akımı miliamperlerden mikroamperlere düşürebilir. Üç temel mod vardır; ne kadar çok kapatırsan o kadar az güç ama o kadar yavaş uyanış.

Mod	Ne kapanır?	Uyanma
Sleep	CPU durur, çevre birimleri çalışır	Herhangi bir kesme
Stop	CPU + saatlerin çoğu durur, RAM korunur	EXTI / RTC alarmı
Standby	Neredeyse her şey kapanır (en düşük güç)	Reset / WKUP pini / RTC

```

1 #include "stm32f1xx_hal.h"
2
3 /* Sleep: en hafif uyku -- kesme gelene kadar CPU'yu durdur (WFI) */
4 void enter_sleep(void)
5 {
6     HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
7     /* Bir kesme gelince buradan devam eder */
8 }
9
10 /* Stop: düşük güç; EXTI (ör. buton) veya RTC alarmı ile uyanır */
11 void enter_stop(void)
12 {
13     HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
14     SystemClock_Config(); /* uyandıktan sonra saati YENİDEN kur */
15 }
16
17 /* Standby: en düşük güç; sadece reset/WKUP/RTC ile uyanır, RAM kaybolur */
18 void enter_standby(void)
19 {
20     HAL_PWR_EnterSTANDBYMode(); /* uyanınca sistem baştan başlar */
21 }

```

Dikkat

Stop modundan sonra saati yeniden kur! Stop modu PLL ve yüksek hızlı saatleri kapattığından, uyandıktan sonra MCU varsayılan yavaş HSI ile çalışır; `SystemClock_Config()`'i tekrar çağırılmazsa çevre birimleri yanlış frekansta çalışır. Standby modunda ise cihaz uyanınca main'den baştan başlar (RAM içeriği kaybolur) — durumu korumak için backup register'ları kullanılır.

İpucu

Tipik düşük güçlü IoT kalıbı: MCU çoğu zaman *Stop* veya *Standby*'da uyur, RTC alarmı (ör. her 10 dakika) onu uyandırır, bir sensör okur, veriyi gönderir ve tekrar uyur. Bu sayede bir pil aylarca hatta yıllarca dayanabilir. Güç ölçümü için bir ampermetre veya ST'nin *Power Shield*'ı kullanılır.

17. CRC ve Diğer Sistem Birimleri

STM32, veri bütünlüğünü donanımda hızlıca doğrulayan bir *CRC* (Cyclic Redundancy Check) birimi içerir. İletişim protokollerinde ve firmware doğrulamada verinin bozulmadığını kontrol etmek için kullanılır.

```

1 #include "stm32f1xx_hal.h"
2
3 CRC_HandleTypeDef hcrc;
4

```

```

5 void crc_hal_init(void)
6 {
7     __HAL_RCC_CRC_CLK_ENABLE();
8     hcrc.Instance = CRC;
9     HAL_CRC_Init(&hcrc);
10 }
11
12 uint32_t compute_crc(uint32_t *data, uint32_t length)
13 {
14     /* Donanım CRC'yi anında hesaplar (yazılım döngüsünden çok hızlı) */
15     return HAL_CRC_Calculate(&hcrc, data, length);
16 }

```

Bilgi

STM32 ailesine göre başka sistem birimleri de bulunur: **DAC** (dijital-analog çevirici — ses/dalga üretimi), **RNG** (donanım rastgele sayı üretici — kriptografi), **FSMC** (harici bellek/ekran arayüzü) ve **option bytes** (Flash koruma, önyükleme seçimi). Hepsi CMSIS/HAL ile erişilebilir; hangi çevre birimlerinin mevcut olduğu kartının datasheet'inde listelenir.

18. Hata Ayıklama ve İzleme (Debugging)

Gömülü kodda hata ayıklamak, PC'dekinden farklıdır: bir ekran veya konsol yoktur. STM32, donanım hata ayıklama için *SWD* (Serial Wire Debug) arayüzü sunar — sadece iki telle (SWDIO, SWCLK) kesme noktası koyma, adım adım ilerleme ve değişken inceleme yapılır.

Yöntem	Kullanım
SWD + GDB	Kesme noktası, adımlama, register/bellek inceleme (en güçlü).
SWO / ITM	Tek telle “printf” izleme (trace), CPU'yu durdurmadan.
UART printf	Basit mesaj çıktısı (ekstra pin gerekir).
LED / GPIO	En ilkel ama bazen en hızlı (bir pini yakıp söndür).

```

1 # OpenOCD ile GDB sunucusu başlat (ST-Link üzerinden)
2 $ openocd -f interface/stlink.cfg -f target/stm32f1x.cfg
3
4 # Başka terminalde GDB ile bağlan
5 $ arm-none-eabi-gdb firmware.elf
6 (gdb) target remote localhost:3333
7 (gdb) load                                # firmware'i karta yükle
8 (gdb) break main                          # main'e kesme noktası
9 (gdb) continue                            # çalıştır
10 (gdb) print my_variable                  # değişken değerini gör
11 (gdb) step                              # tek satır ilerle

```

18.1 SWO ile printf İzleme (ITM)

SWO (Serial Wire Output), printf çıktısını tek bir izleme teli üzerinden gönderir — UART pini harcamadan ve CPU'yu neredeyse hiç yavaşlatmadan.

```

1 #include "stm32f1xx.h"
2
3 /* printf'i ITM (SWO) üzerinden gönder */
4 int _write(int file, char *ptr, int len)
5 {
6     for (int i = 0; i < len; i++) {
7         ITM_SendChar(*ptr++);      /* CMSIS: karakteri ITM stimulus portuna yaz */
8     }
9     return len;
10 }
11 /* PC'de: st-link'in SWO görüntüleyicisi veya "openocd + itmdump" ile okunur */

```

18.2 HardFault Ayıklama

Bir program çökerse (geçersiz bellek erişimi, hizalanmamış erişim, sıfıra bölme) çekirdek HardFault_Handler'atlar. Buradaki fault register'larını inceleyerek çökme sebebini bulabilirsin.

```

1 /* HardFault işleyici: çökme anındaki durumu incele */
2 void HardFault_Handler(void)
3 {
4     /* SCB->CFSR: hangi fault türü (bus/usage/memory) olduğunu söyler */
5     volatile uint32_t cfsr = SCB->CFSR;      /* Configurable Fault Status Reg */
6     volatile uint32_t hfsr = SCB->HFSR;      /* HardFault Status Reg */
7     volatile uint32_t faddr = SCB->BFAR;      /* hataya neden olan adres */
8     (void)cfsr; (void)hfsr; (void)faddr;
9
10    while (1);                                /* debugger ile buraya bakılır */
11 }

```

İpucu

Bir HardFault yakaladığında, hata ayıklayıcıyla durup CFSR register'ını (adres 0xE000ED28) inceleyerek türü (bus fault, usage fault...), BFAR/MMFAR ile de hatalı adresi öğrenebilirsin. Ayrıca yığına itilmiş PC değeri (çökme anındaki komut) çökmenin *tam satırını* verir. arm-none-eabi-addr2line ile o adresi kaynak satırına çevir.

19. FreeRTOS: Gerçek Zamanlı İşletim Sistemi

Bir uygulama büyüdüğünde, tek bir while(1) döngüsünde her şeyi yönetmek zorlaşır. RTOS (Real-Time Operating System), programı bağımsız *görevlere* (task) böler ve bir *zamanlayıcı* (scheduler) onları sırayla/önceliğe göre çalıştırır — her biri ayrı bir while(1) gibi. FreeRTOS, STM32'de en yaygın RTOS'tur ve CubeMX ile hazır gelir.

19.1 Görevler (Tasks)

```

1 #include "FreeRTOS.h"
2 #include "task.h"
3
4 /* Her görev sonsuz döngülü bir fonksiyondur, asla return etmez */
5 void led_task(void *params)
6 {
7     while (1) {
8         HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
9     }
10 }

```



```

9      vTaskDelay(pdMS_TO_TICKS(500));    /* 500 ms uyu -- CPU'yu bırakır! */
10  }
11 }
12
13 void sensor_task(void *params)
14 {
15     while (1) {
16         read_sensor();
17         vTaskDelay(pdMS_TO_TICKS(100));
18     }
19 }
20
21 int main(void)
22 {
23     HAL_Init();
24     /* ... donanım kurulumu ... */
25
26     /* Görevleri oluştur (fonksiyon, ad, yığın boyutu, param, öncelik, handle) */
27     xTaskCreate(led_task, "LED", 128, NULL, 1, NULL);
28     xTaskCreate(sensor_task, "Sensor", 256, NULL, 2, NULL);
29
30     vTaskStartScheduler();                /* zamanlayıcıyı başlat -- geri dönmez */
31     while (1);
32 }

```

vTaskDelay vs meşgul bekleme

Kritik fark: vTaskDelay, görevi uyuttuğunda CPU'yu *başka görevlere* bırakır — meşgul döngü gibi boşa harcamaz. Zamanlayıcı, uyuyan görev beklerken diğerlerini çalıştırır. Bu sayede birden çok iş, tek çekirdekte eşzamanlı gibi yürür.

19.2 Görevler Arası İletişim: Kuyruk ve Semafor

Görevler veriyi güvenli paylaşmak için *kuyruk* (queue), senkronizasyon için *semafor*/*mutex* kullanılır. Bunlar yarış koşullarını önler.

Nesne

Kullanım

Queue	Görevler/ISR arası veri aktarımı (thread-safe FIFO).
Semaphore	Olay bildirimi / kaynak sayımı (ör. ISR görevi uyandırır).
Mutex	Paylaşılan kaynağı (UART, SPI) tek görevin kullanmasını sağlar.

```

1  #include "queue.h"
2
3  QueueHandle_t data_queue;
4
5  void producer_task(void *params)
6  {
7      uint32_t value = 0;
8      while (1) {
9          value = read_sensor();
10         xQueueSend(data_queue, &value, portMAX_DELAY);    /* kuyruğa koy */
11         vTaskDelay(pdMS_TO_TICKS(100));
12     }

```

```
13 }
14
15 void consumer_task(void *params)
16 {
17     uint32_t received;
18     while (1) {
19         /* Veri gelene kadar UYU (CPU harcamaz), gelince işle */
20         if (xQueueReceive(data_queue, &received, portMAX_DELAY) == pdTRUE) {
21             process(received);
22         }
23     }
24 }
25
26 /* main içinde: data_queue = xQueueCreate(10, sizeof(uint32_t)); */
```

Bilgi

FreeRTOS'un ISR ile birlikte çalışan sürümleri vardır (...FromISR): bir kesme, xQueueSendFromISR veya xSemaphoreGiveFromISR ile bir görevi uyandırabilir. Böylece kesme işleyicisi kısa kalır (sadece bildirir), asıl işi bir görev yapar — Kısım 3'teki "üst/alt yarı" fikrinin RTOS karşılığı. FreeRTOS, arka planda Kısım 8'de göreceğimiz PendSV kesmesiyle görevler arası geçiş yapar.

KISIM 6

ARM Cortex-M Assembly Temelleri

Şimdiye kadar STM32'yi C ile programladık. Artık bir kat aşağıya — C kodunun çevrildiği *assembly* diline — iniyoruz. Assembly bilmek; startup kodunu anlamak, hata ayıklamada disassembly okumak, kritik döngüleri optimize etmek ve donanımın gerçekte nasıl çalıştığını kavramak için gereklidir.

20. ARM Mimarisi ve Programlama Modeli

20.1 RISC ve Load/Store Mimarisi

ARM bir *RISC* (Reduced Instruction Set Computer) mimarisidir: az sayıda, basit ve sabit boyutlu komutları vardır. En önemli özelliği *load/store* mimarisi olmasıdır: aritmetik/mantık işlemleri **yalnızca register'lar** üzerinde yapılır; bellek yalnızca özel LDR (load) ve STR (store) komutlarıyla okunur/yazılır.

	ARM (RISC)	x86 (CISC)
Komut sayısı	Az, basit	Çok, karmaşık
Bellek işlemi	Sadece LDR/STR	Çoğu komut belleğe erişebilir
Komut boyutu	Sabit (2/4 bayt)	Değişken (1–15 bayt)
Güç verimi	Yüksek (gömülü için ideal)	Düşük

20.2 Register'lar (Cortex-M Programlama Modeli)

Cortex-M'de 16 adet 32-bit register vardır. R0–R12 genel amaçlıdır; R13–R15'in özel görevleri vardır.

Register	Ad	Görev
R0–R12	Genel amaçlı	Veri ve ara hesaplamalar.
R13	SP (Stack Pointer)	Yığın tepesini gösterir.
R14	LR (Link Register)	Fonksiyon dönüş adresini tutar.
R15	PC (Program Counter)	Çalışan komutun adresi.
--	xPSR	Durum bayrakları (N, Z, C, V) ve durum.

20.3 Durum Bayrakları (Condition Flags)

Aritmetik ve karşılaştırma komutları, sonuç hakkında *bayrakları* (APSR içinde) günceller. Dalanma (branch) kararları bu bayraklara bakar.

Bayrak	Ad	Anlamı
N	Negative	Sonuç negatif (en yüksek bit 1).
Z	Zero	Sonuç sıfır.
C	Carry	Elde/ödünç oluştu (taşma, işaretli).
V	Overflow	İşaretli taşma.

20.4 Thumb ve Thumb-2

Klasik ARM komutları 32-bit'tir. *Thumb*, aynı işleri 16-bit komutlarla yaparak kod boyutunu küçültür. *Thumb-2*, 16 ve 32-bit komutları karıştırıp hem küçük hem güçlü olur. **Cortex-M yalnızca Thumb-2 kullanır** — bu yüzden STM32 assembly'sinde ARM state yoktur; her şey Thumb'dır.

Bilgi

Bu rehberde **GNU assembler** (arm-none-eabi-as) sözdizimini kullanacağız. Dosyanın başında `.syntax unified` (birleşik sözdizimi) ve `.thumb` (Thumb-2 modu) yönergeleri bulunur. Yorumlar `;` veya `//` ile, etiketler `label:` biçiminde yazılır.

21. Temel Komutlar: Veri ve Aritmetik

21.1 Veri Taşıma: MOV ve LDR

MOV, register'lar arasında veya küçük bir sabiti bir register'a taşır. Büyük 32-bit sabitler tek komuta sığmaz; onlar için MOVW/MOVT çifti veya LDR Rd, =sabit sözde-komutu (literal pool) kullanılır.

```

1  MOV    r0, #5           ; r0 = 5 (küçük sabit doğrudan)
2  MOV    r1, r0           ; r1 = r0 (register kopyala)
3  MOVW   r2, #0x1234      ; r2'nin alt 16 biti = 0x1234
4  MOVT   r2, #0x5678      ; r2'nin üst 16 biti = 0x5678 -> r2 = 0x56781234
5  LDR    r3, =0x40011000  ; büyük sabiti yükle (literal pool ile) -- GPIOC adresi

```

21.2 Bellek Erişimi: LDR ve STR

Load/store mimarisinin kalbi: LDR bellekten register'a *yükler*, STR register'dan belleğe *yazar*. Bayt için LDRB/STRB, yarım kelime için LDRH/STRH kullanılır.

```

1  LDR    r1, =myvar       ; r1 = myvar'ın ADRESİ
2  LDR    r0, [r1]         ; r0 = *r1 (adresteki değeri yükle)
3  ADD    r0, r0, #1       ; r0 = r0 + 1
4  STR    r0, [r1]         ; *r1 = r0 (değeri belleğe geri yaz)
5
6  LDRB   r2, [r1]         ; tek bayt yükle (alt 8 bit)
7  STRB   r2, [r1, #4]     ; r1+4 adresine bayt yaz (offset ile)

```

Dikkat

Aritmetik komutlar (ADD, SUB...) **belleğe doğrudan erişemez**. Bir bellek konumunu artırmak üç adım gerektirir: LDR (yükle) → ADD (değiştir) → STR (geri yaz). Bu, x86'dan gelenler için ilk şaşırtıcı noktadır; ama load/store mimarisinin özüdür.

21.3 Aritmetik ve Mantık Komutları

Komut	İşlevi
ADD Rd, Rn, Rm	$Rd = Rn + Rm$ (topla)
SUB Rd, Rn, Rm	$Rd = Rn - Rm$ (çıkar)
MUL Rd, Rn, Rm	$Rd = Rn \times Rm$ (çarp)
UDIV / SDIV	Bölme (işaretsiz / işaretli)
AND / ORR / EOR	Bit bazlı VE / VEYA / ÖZEL-VEYA (XOR)
BIC	Bit temizle ($Rn \text{ AND NOT } Rm$)
LSL / LSR	Sola / sağa mantıksal kaydır

```

1  ADD    r0, r1, r2      ; r0 = r1 + r2
2  SUB    r0, r0, #10     ; r0 = r0 - 10 (sabit ile)
3  MUL    r3, r1, r2      ; r3 = r1 * r2
4  ADDS   r0, r1, r2      ; topla VE bayrakları güncelle (S soneki)
5
6  ORR    r0, r0, #(1 << 13) ; r0'ın 13. bitini kur (register set örneği)
7  BIC    r0, r0, #(1 << 13) ; r0'ın 13. bitini temizle
8  LSL    r0, r1, #2      ; r0 = r1 << 2 (4 ile çarpma)

```

“S” soneki

ADD bayrakları değiştirmez, ama ADDS (sondaki S) sonuç hakkında N/Z/C/V bayraklarını günceller. Bir sonraki bölümdeki koşullu dallanmalar bu bayraklara baktığından, karşılaştırma/döngü kodunda çoğu zaman S'li sürümler veya CMP kullanılır.

KISIM 7

ARM Assembly: Akış, Fonksiyonlar ve Uygulama

22. Akış Kontrolü: Dallanma ve Döngüler

Yüksek seviyeli dillerdeki if, while, for yapıları assembly'de *karşılaştırma + koşullu dallanma* (branch) ikiliyle kurulur. Önce CMP ile bayraklar ayarlanır, sonra koşullu bir B komutu bayraklara bakarak atlar.

22.1 Karşılaştırma ve Koşullu Dallanma

CMP r0, r1 aslında r0 - r1 yapar ama sonucu *saklamaz*; yalnızca bayrakları günceller. Ardından koşul ekli bir dal komutu gelir:

Komut	Koşul	Komut	Koşul
BEQ	eşit (=)	BNE	eşit değil ($\neq$)
BGT	büyük (>, işaretli)	BLT	küçük (<, işaretli)
BGE	büyük/eşit ($\geq$)	BLE	küçük/eşit ($\leq$)
BHI	büyük (işaretsiz)	BLO	küçük (işaretsiz)
B	koşulsuz (her zaman)	BX	register'a dallan (dönüş)

22.2 if Yapısı

```

1      ; C karşılığı: if (r0 > 10) { r1 = 1; } else { r1 = 0; }
2      CMP    r0, #10          ; r0 ile 10'u karşılaştır (bayrakları ayarla)
3      BLE    else_branch      ; r0 <= 10 ise else'e atla
4      MOV    r1, #1          ; if bloğu: r1 = 1
5      B      end_if          ; else'i atla
6  else_branch:
7      MOV    r1, #0          ; else bloğu: r1 = 0
8  end_if:
9      ; devam...

```

22.3 Döngü: Bir Diziyi Toplama

```

1      ; C karşılığı: sum = 0; for (i=0; i<n; i++) sum += array[i];
2      LDR    r1, =data_array ; r1 = dizinin adresi
3      MOV    r2, #0          ; r2 = sum = 0
4      MOV    r3, #0          ; r3 = i = 0
5      MOV    r4, #5          ; r4 = n = 5 (eleman sayısı)
6      loop_start:
7          CMP    r3, r4      ; i < n mi?
8          BGE    loop_end    ; i >= n ise döngüden çık
9          LDR    r5, [r1, r3, LSL #2] ; r5 = array[i] (i*4 offset ile)
10         ADD    r2, r2, r5    ; sum += array[i]
11         ADD    r3, r3, #1    ; i++
12         B      loop_start    ; döngü başına dön
13      loop_end:
14         ; r2 artık toplamı içerir

```

Bilgi

LDR r5, [r1, r3, LSL #2] tek bir komutta güçlü bir iş yapar: r3'ü (indeks) 2 bit sola kaydırıp (yani 4 ile çarpıp — int boyutu) r1'e (taban adres) ekler ve o adresteki değeri yükler. Yani array[i]'yi tek komutta okur. Buna *ölçekli register offset* adresleme denir — ARM'ın verimliliğinin bir örneği.

23. Fonksiyonlar, Yığın ve Çağrı Kuralı (AAPCS)

Fonksiyon çağırmak, BL (Branch with Link) ile yapılır: hedefe atlar ve dönüş adresini LR'ye kaydeder. Fonksiyondan BX LR ile dönlür. Ama fonksiyonların birlikte çalışabilmesi için herkesin uyduğu bir sözleşme gerekir: *AAPCS* (ARM Architecture Procedure Call Standard).

23.1 Çağrı Kuralı (Calling Convention)

Register	Rol
R0-R3	İlk 4 argüman; ve scratch (çağırana korumaz). R0 = dönüş değeri.
R4-R11	Yerel değişkenler; callee-saved (kullanan korumalı).
R12 (IP)	Scratch (ara).
LR (R14)	Dönüş adresi.
SP (R13)	Yığın işaretçisi; 8 bayt hizalı tutulmalı.

23.2 Basit Fonksiyon (Leaf Function)

Başka fonksiyon çağırmayan (leaf) bir fonksiyon, LR'yi saklamaya gerek duymaz — doğrudan BX LR ile döner.

```

1      ; int add(int a, int b) { return a + b; }
2      ; a -> r0, b -> r1, dönüş -> r0
3      add:
4          ADD    r0, r0, r1    ; r0 = a + b (dönüş değeri r0'da)
5          BX     lr           ; çağırana dön

```

23.3 Yığın (Stack): PUSH ve POP

Bir fonksiyon başka fonksiyon çağıracaksa, BL LR'nin üzerine yazacağı için önce LR'yi *yığına* kaydetmelidir. Ayrıca callee-saved register'ları (R4-R11) kullanıyorsa onları da saklamalıdır. Yığın *full-descending*'dir: aşağı doğru büyür.

```

1      ; int compute(int x) { return add(x, 10) * 2; }
2 compute:
3      PUSH {r4, lr}      ; r4 ve LR'yi yığına kaydet (çağrı LR'yi bozacak)
4      MOV  r4, r0        ; x'i r4'te sakla (add çağrısı r0'ı bozacak)
5      MOV  r1, #10
6      BL   add           ; r0 = add(x, 10) -- LR değişir, r0-r3 bozulur
7      LSL  r0, r0, #1     ; r0 = sonuç * 2
8      POP  {r4, pc}      ; r4'ü geri yükle; LR'yi PC'ye yükleyerek DÖN

```

POP {..., pc} hilesi

Dönüş adresini LR'ye geri yükleyip sonra BX LR yapmak yerine, PUSH {..., lr} ile kaydedilen dönüş adresini doğrudan POP {..., pc} ile PC'ye yüklemek — tek komutta hem register'ları geri yükler hem de fonksiyondan döner. Bu, ARM kodunda çok yaygın ve verimli bir kalıptır.

23.4 Özyinelemeli Fonksiyon: Faktöriyel

```

1      ; int factorial(int n) { return n<=1 ? 1 : n*factorial(n-1); }
2 factorial:
3      PUSH {r4, lr}      ; n'i ve dönüş adresini koru
4      CMP  r0, #1        ; n <= 1 mi?
5      BGT  recurse      ; n > 1 ise özyinele
6      MOV  r0, #1        ; taban durum: return 1
7      POP  {r4, pc}
8 recurse:
9      MOV  r4, r0        ; n'i sakla
10     SUB  r0, r0, #1     ; r0 = n - 1
11     BL   factorial     ; r0 = factorial(n-1)
12     MUL  r0, r4, r0     ; r0 = n * factorial(n-1)
13     POP  {r4, pc}      ; geri yükle ve dön

```

24. Donanım, Startup ve C ile Karıştırma

24.1 Assembly'den GPIO Register'ına Yazma

Kısım 2'deki LED yakmayı, şimdi doğrudan assembly ile yapalım. Sadece register adresine bir değer yazmaktan ibarettir.

```

1      ; GPIOC->BSRR = (1 << 13); -- PC13 LED'ini yak (register seviyesi)
2      LDR  r0, =0x40011010 ; r0 = GPIOC_BSRR adresi (BSRR ofseti 0x10)
3      MOV  r1, #(1 << 13) ; r1 = PC13 için bit maskesi
4      STR  r1, [r0]       ; BSRR'ye yaz -> pin atomik olarak set edilir

```

24.2 C ile Inline Assembly

C kodunun içine, GCC'nin asm sözdizimiyle assembly gömebiliriz. Kesme kapatma/açma gibi çekirdek işlemler genelde böyle yapılır.


```

1  /* Cortex-M çekirdek komutlarını C'den çağırma (inline assembly) */
2  static inline void disable_irq(void)
3  {
4      __asm__ volatile ("cpsid i");    /* tüm kesmeleri kapat (interrupt disable) */
5  }
6
7  static inline void enable_irq(void)
8  {
9      __asm__ volatile ("cpsie i");    /* kesmeleri aç */
10 }
11
12 static inline void wait_for_interrupt(void)
13 {
14     __asm__ volatile ("wfi");        /* kesme gelene kadar uyu (düşük güç) */
15 }

```

24.3 Cortex-M Startup Kodu ve Vektör Tablosu

Reset sonrası ilk çalışan kod, assembly ile yazılmış *startup* kodudur. Cortex-M'de vektör tablosunun ilk iki girdisi özeldir: başlangıç yığın işaretçisi (SP) ve reset işleyicisinin (ilk çalışacak kod) adresi.

```

1  .syntax unified
2  .thumb
3
4  .section .isr_vector, "a"      ; vektör tablosu (Flash başında)
5  .word  _stack_top             ; [0] başlangıç SP değeri
6  .word  Reset_Handler         ; [1] reset sonrası çalışacak adres
7  .word  NMI_Handler           ; [2] NMI
8  .word  HardFault_Handler     ; [3] hard fault
9  ; ... diğer işleyiciler ...
10
11 .section .text
12 .thumb_func
13 Reset_Handler:
14     LDR    sp, =_stack_top     ; yığın işaretçisini kur
15     BL     main                ; C main() fonksiyonuna atla
16 hang:
17     B      hang                ; main dönerse burada takıl (sonsuz döngü)

```

Bilgi

Gerçek startup kodu ek işler de yapar: `.data` bölümünü Flash'tan RAM'e kopyalamak (iklendirilmiş global değişkenler) ve `.bss` bölümünü sıfırlamak (iklendirilmemiş globaller). Bunlar C'nin `main`'den önce beklediği ortamı hazırlar. STM32CubeIDE bu startup dosyasını (`startup_stm32f103.s`) otomatik üretir; ama içinde ne olduğunu bilmek, gömülü sistemi gerçekten anlamak demektir.

KISIM 8

ARM Assembly: İleri Konular

Bu kısım, Cortex-M assembly'sinin daha güçlü özelliklerini kapsar: tek komutta kaydırma yapan barrel shifter, zengin adresleme modları, atomik erişim, özel register'lar, kayan nokta (FPU) ve donanım exception'larını assembly ile işleme. Bunlar, verimli sürücüler, işletim sistemi çekirdekleri ve gerçek zamanlı kod yazmanın anahtarıdır.

25. Barrel Shifter ve Adresleme Modları

25.1 Barrel Shifter: Bedava Kaydırma

ARM'ın ayırt edici özelliği *barrel shifter*'dir: birçok komutun ikinci operandı, *aynı komut içinde* ekstra çevrim harcamadan kaydırılabilir. Böylece “4 ile çarp ve topla” gibi işlemler tek komutta yapılır.

```

1  ADD    r0, r1, r2, LSL #2    ; r0 = r1 + (r2 << 2) yani r1 + r2*4
2  MOV    r0, r1, LSR #1       ; r0 = r1 >> 1 (2'ye bölme)
3  ADD    r0, r1, r1, LSL #3    ; r0 = r1 + r1*8 = r1*9 (çarpmasız!)
4  RSB    r0, r1, r1, LSL #4    ; r0 = r1*16 - r1 = r1*15

```

Bilgi

Barrel shifter, derleyicilerin sabitle çarpmayı neden nadiren gerçek MUL ile yaptığını açıklar: $x*9$ yerine `ADD r0, r1, r1, LSL #3` tek çevrimde biter. Kaydırma türleri: LSL (mantıksal sol), LSR (mantıksal sağ), ASR (aritmetik sağ — işaret korur), ROR (döndür).

25.2 Adresleme Modları

LDR/STR komutlarının adres hesaplama biçimleri (addressing modes) zengindir. Özellikle *ön/son indeksleme*, bir işaretçiyi ilerletirken belleğe erişmeyi tek komutta yapar — dizi/tampon döngülerinde çok verimli.

Mod	Anlamı
[r1]	Adres = r1 (doğrudan).
[r1, #4]	Adres = r1 + 4 (offset; r1 değişmez).
[r1, r2]	Adres = r1 + r2 (register offset).
[r1, r2, LSL #2]	Adres = r1 + r2*4 (ölçekli — array[i]).
[r1, #4]!	Ön-indeks: r1 += 4, sonra eriş (r1 güncellenir).
[r1], #4	Son-indeks: eriş, sonra r1 += 4.

```

1      ; Bir diziyi son-indeksleme ile gez (işaretçi otomatik ilerler)
2      LDR    r1, =buffer          ; r1 = tampon başlangıcı
3 copy_loop:
4      LDR    r0, [r1], #4          ; r0 = *r1; sonra r1 += 4 (bir sonraki word)
5      ; ... r0'ı işle ...
6      ; döngü koşulu ile copy_loop'a dön

```

25.3 LDM / STM: Çoklu Yükleme/Saklama

Tek komutla birden çok register'ı arka arkaya belleğe yazabilir veya okuyabiliriz. Bu, blok kopyalama ve yığın işlemlerinin (PUSH/POP aslında STMDB/LDMIA'dır) temelidir.

```

1      ; 8 word'ü tek komutta kopyala (blok transfer)
2      LDMIA r0!, {r4-r7}          ; r0'dan 4 word yükle -> r4,r5,r6,r7; r0 += 16
3      STMIA r1!, {r4-r7}          ; r1'e 4 word yaz; r1 += 16
4      ; IA = Increment After (adres arttıksa); PUSH = STMDB sp!, POP = LDMIA sp!

```

26. Koşullu Yürütme ve Bit Komutları

26.1 IT Blokları (If-Then)

Thumb-2'de, dallanma yapmadan bir-dört komutu koşullu çalıştırmak için *IT* (If-Then) bloğu kullanılır. Kısa if yapılarında dal maliyetinden (pipeline flush) kaçınır.

```

1      ; if (r0 == 0) r1 = 1; else r1 = 2;
2      CMP    r0, #0
3      ITE    EQ                    ; If-Then-Else: sonraki EQ, ondan sonraki NE
4      MOVEQ  r1, #1                ; r0 == 0 ise çalışır
5      MOVNE  r1, #2                ; r0 != 0 ise çalışır

```

26.2 Bit Alanı ve Sıralama Komutları

Cortex-M, bit manipülasyonu için özel komutlar sunar — gömülü register işlemede çok kullanışlıdır.

Komut	İşlevi
UBFX / SBFX	Bir bit alanını çıkar (unsigned/signed bit field extract).
BFI / BFC	Bit alanı ekle / temizle (insert/clear).
CLZ	Baştaki sıfır bitleri say (count leading zeros).
RBIT	Bit sırasını tersine çevir.
REV / REV16	Bayt sırasını tersle (endian dönüşümü).

```

1  UBFX  r0, r1, #4, #3      ; r1'in 4. bitinden başlayan 3 biti r0'a çıkar
2  BFI   r1, r0, #8, #4      ; r0'ın alt 4 bitini r1'in 8. bitine yerleştir
3  CLZ   r0, r1              ; r1'de baştaki sıfır sayısı (öncelik kodlama)
4  REV   r0, r1              ; bayt sırasını tersle (big<->little endian)

```

27. Özel Register'lar, Atomik Erişim ve Bariyerler

27.1 Özel Register Erişimi: MRS ve MSR

Genel register'ların dışındaki *özel* register'lara (durum, kesme maskesi, yığın işaretçileri) yalnızca MRS (oku) ve MSR (yaz) komutlarıyla erişilir. RTOS ve çekirdek kodu bunları yoğun kullanır.

Register	İşlevi
PRIMASK	Tüm kesmeleri kapat/aç (cpsid/cpsie i bunu değiştirir).
BASEPRI	Belirli öncelik altındaki kesmeleri maskele.
CONTROL	Ayrıcalık seviyesi ve MSP/PSP seçimi.
MSP / PSP	Main / Process yığın işaretçileri (RTOS ayrımı).

```

1  MRS   r0, PRIMASK        ; mevcut kesme maskesini oku (kaydet)
2  CPSID i                  ; tüm kesmeleri kapat (kritik bölge)
3  ; ... kritik kod ...
4  MSR   PRIMASK, r0        ; önceki maske durumunu geri yükle
5
6  MRS   r0, PSP            ; process yığın işaretçisini oku (RTOS bağlam değiştirme)

```

27.2 Atomik Erişim: LDREX ve STREX

Çok görevli veya kesme paylaşımlı kodda “oku-değiştir-yaz” atomik olmalıdır. LDREX/STREX çifti bunu sağlar: LDREX özel bir işaretle okur, STREX yalnızca arada kimse yazmadıysa başarılı olur. RTOS kilitlerinin (mutex, semafor) temelidir.

```

1  ; Atomik artırma: *r0 += 1 (kesintiye dayanıklı)
2  atomic_inc:
3  LDREX r1, [r0]           ; değeri özel (exclusive) olarak oku
4  ADD   r1, r1, #1         ; artır
5  STREX r2, r1, [r0]       ; geri yaz; r2=0 başarı, r2=1 başarısız
6  CMP   r2, #0             ; başarısızsa (araya girildiyse)...

```

```

7   BNE    atomic_inc      ; ...baştan dene
8   BX     lr

```

27.3 Bellek Bariyerleri: DMB, DSB, ISB

Cortex-M işlemcileri bellek erişimlerini yeniden sıralayabilir. Belirli sıralamanın garanti edilmesi gereken yerlerde (çevre birimi yapılandırması, bağlam değiştirme) *bellek bariyerleri* kullanılır.

Bariyer	Garantisi
DMB	Data Memory Barrier: önceki bellek erişimleri sonrakilerden önce görünür.
DSB	Data Sync Barrier: önceki tüm erişimler tamamlanmadan devam etmez.
ISB	Instruction Sync Barrier: pipeline’ı temizler (yeni ayarları uygula).

Ne zaman gerekir?

Günlük C kodunda nadiren; ama bir çevre biriminin saatini açtıktan hemen sonra register’ına yazarken, veya vektör tablosunu değiştirdikten sonra DSB/ISB gerekebilir. CMSIS bunları `__DMB()`, `__DSB()`, `__ISB()` intrinsic’leriyle sunar — assembly yazmadan C’den çağırabilirsin.

28. FPU ve Assembly’de Exception İşleme

28.1 Kayan Nokta (FPU) Komutları

Cortex-M4F ve M7 çekirdekleri bir donanım *FPU* (Floating-Point Unit) içerir; kayan nokta işlemlerini yazılım öykünmesinden çok daha hızlı yapar. FPU komutları V ile başlar ve ayrı S0–S31 register’larını kullanır.

```

1   VLDR    s0, [r0]        ; bellekten float yükle -> s0
2   VLDR    s1, [r1]
3   VADD.F32 s2, s0, s1     ; s2 = s0 + s1 (tek hassasiyetli float toplama)
4   VMUL.F32 s2, s2, s0     ; s2 = s2 * s0
5   VSTR     s2, [r2]       ; sonucu belleğe yaz
6   VCVT.S32.F32 r3, s2    ; float -> int dönüşümü

```

Dikkat

FPU’yu kullanmadan önce **etkinleştirmelisin**: CPACR register’ında CP10/CP11 erişimini açman gerekir (genelde startup kodu yapar). Ayrıca FPU register’ları da bağlam (context) parçasıdır; bir RTOS veya kesme FPU kullanıyorsa, bağlam değiştirmede S0–S31’in de kaydedilmesi gerekir — Cortex-M bunun için “lazy stacking” mekanizması sunar.

28.2 Assembly ile Exception İşleyici

Cortex-M’de bir kesme/exception oluştuğunda, donanım *otomatik olarak* sekiz register’ı (R0–R3, R12, LR, PC, xPSR) yığına iter ve LR’ye özel bir EXC_RETURN değeri koyar. Bu sayede ISR’ler *normal C fonksiyonu* gibi yazılabilir. Assembly ile bir işleyici yazmak, bağlam değiştirme (context switch) gibi işler için gerekir.

```

1  .thumb_func
2  PendSV_Handler:                ; RTOS bağlam değiştirme genelde burada olur
3      MRS    r0, PSP              ; mevcut görevin yığın işaretçisini al
4      STMDB  r0!, {r4-r11}        ; kalan register'ları (donanım R0-R3'ü zaten itti) kaydet
5      ; ... görev yığın işaretçisini kaydet, sonraki görevi yükle ...
6      LDMIA  r0!, {r4-r11}        ; yeni görevin register'larını geri yükle
7      MSR    PSP, r0
8      BX     lr                  ; EXC_RETURN ile normal göreve dön (donanım gerisini yükler)

```

Bilgi

Donanımın otomatik “stacking”i (R0-R3, R12, LR, PC, xPSR’yi itmesi), ISR’leri C’de yazmayı mümkün kılar — çünkü çağırıcı-korumalı (caller-saved) register’lar zaten kaydedilmiştir. EXC_RETURN (LR’deki `0xFFFF_FFFF` değeri), işlemciye hangi yığına (MSP/PSP) döneceğini ve FPU bağlamı olup olmadığını söyler. Bu mekanizma, FreeRTOS gibi çekirdeklerin PendSV ile görevler arası geçiş yapmasının temelidir.

29. Kapanış: Gömülü Geliştirme Yol Haritası

Bu rehberde STM32’yi register seviyesinden başlayıp assembly’ye kadar iki yönde inceledik. İki bakış birbirini tamamlar: C ile hızlı geliştirir, assembly ile altta ne olduğunu anlarsın.

İhtiyaç	Kullanılan mekanizma
Pin sürme/okuma	GPIO register’ları (ODR, IDR, BSRR)
Çevre birimini açma	RCC saat etkinleştirme
Kesin zamanlama / PWM	Timer (PSC, ARR, CCR)
PC ile iletişim	UART
Analog okuma	ADC
Sensör veri yolu	SPI / I2C
Olay tabanlı tepki	Kesme + NVIC (yoklama yerine)
CPU’suz veri taşıma	DMA
Altyapıyı anlama / optimizasyon	ARM Cortex-M assembly

İpucu

Öğrenmenin en iyi yolu **gerçek donanımla pratik yapmaktır**. Ucuz bir “Blue Pill” veya Nucleo kartı al, önce LED yak, sonra buton oku, UART ile PC’ye mesaj gönder, bir kesme kur. Her adımda `arm-none-eabi-objdump -d` ile C kodunun ürettiği assembly’yi incele — bu, iki kısmı birbirine bağlar. Başvuru için: *STM32 Reference Manual* (register detayları), *Cortex-M Generic User Guide* ve *ARMv7-M Architecture Reference Manual* (assembly). Bol dene, boz, hata ayıkla — gömülü sistemi öğrenmenin tek yolu budur. İyi kodlamalar!